Abon, Benedict Aldous A. CPE311 - CPE22S3 03/03/26 03/03/26 Engr. Neil Barton James Matira

Hands-on Activity 9.1 Data Visualization using Pandas and Matplotlib

Getting Started with Matplotlib

Importing Libraries

```
import matplotlib.pyplot as plt
import pandas as pd
```

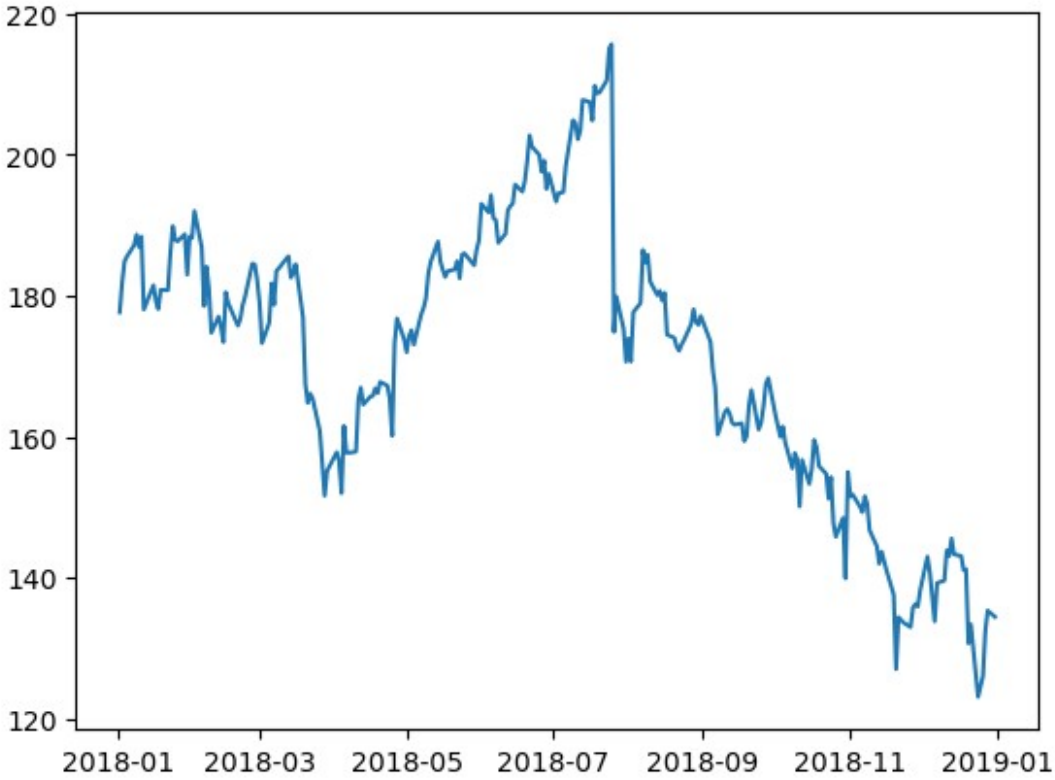
About the Data

In this notebook, we will be working with 2 datasets:

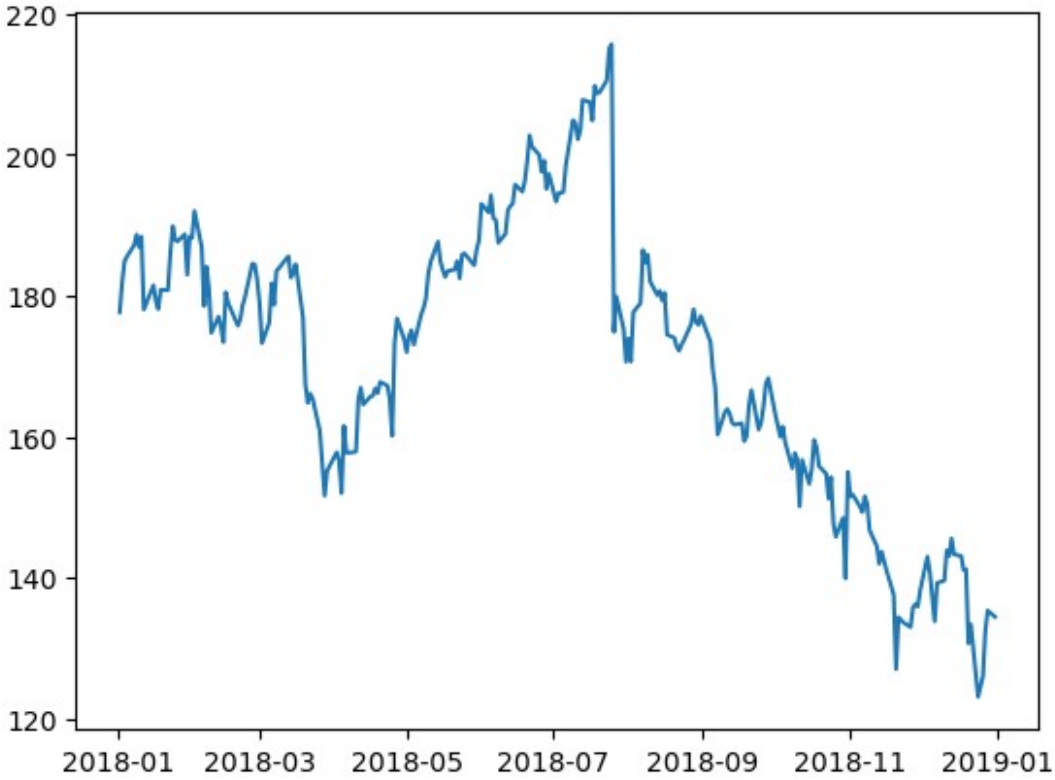
- Facebook's stock price throughout 2018 (obtained using the stock_analysis package)
- Earthquake data from September 18, 2018 - October 13, 2018 (obtained from the US Geological Survey (USGS) using the USGS API)

Plotting Lines

```
fb = pd.read_csv('fb_stock_prices_2018.csv', index_col='date',
parse_dates=True)

plt.plot(fb.index, fb.open)
plt.show()
```




```
%matplotlib inline
plt.plot(fb.index, fb.open)
plt.show()
```

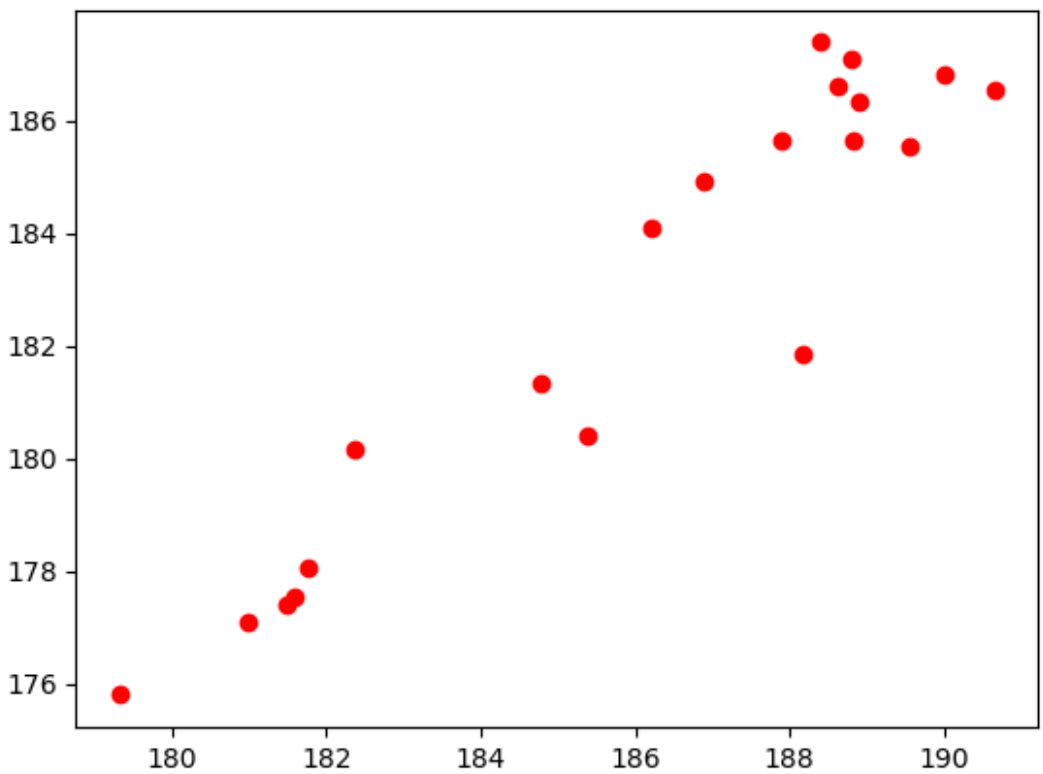


Analysis

The line plot can simply be made by using plt.plot() and passing the x and y values. The line plot shows that Facebook's stock price had a general upward trend throughout 2018, with some fluctuations along the way.

Scatter Plots

```
plt.plot('high', 'low', 'ro', data=fb.head(20))
plt.show()
```

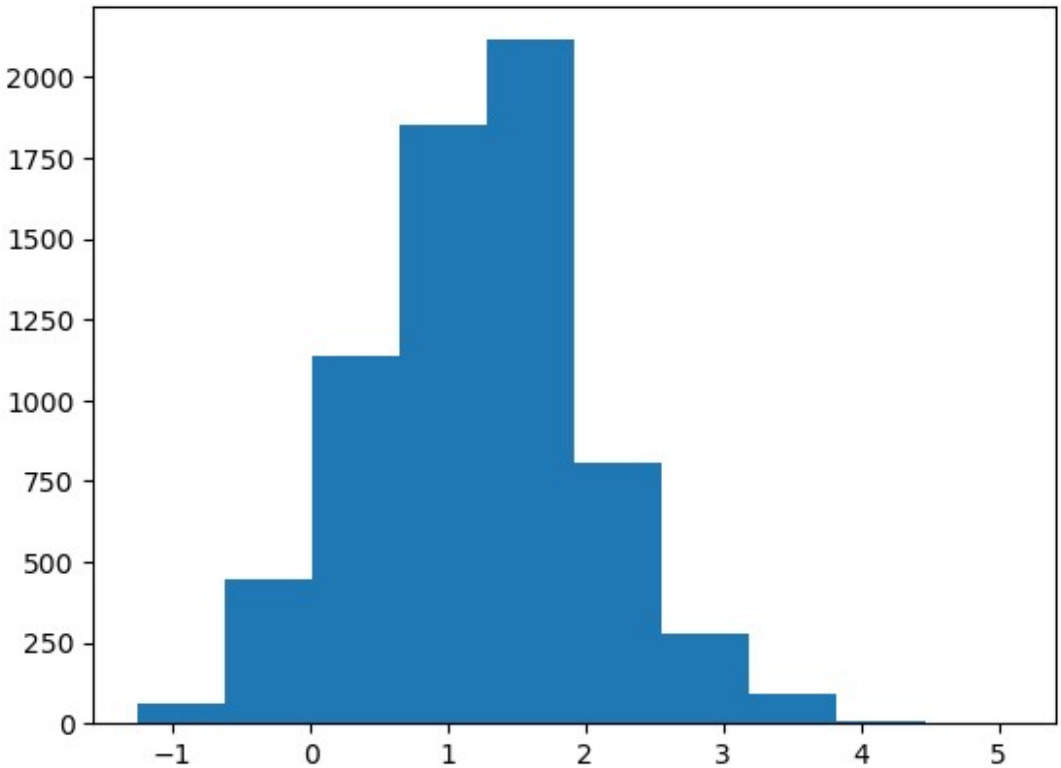


Analysis

The scatter plot can be made by also using plt.plot() but with a different marker. The scatter plot shows the correlation between the high and low stock prices of Facebook. The points indicate a positive correlation between variables.

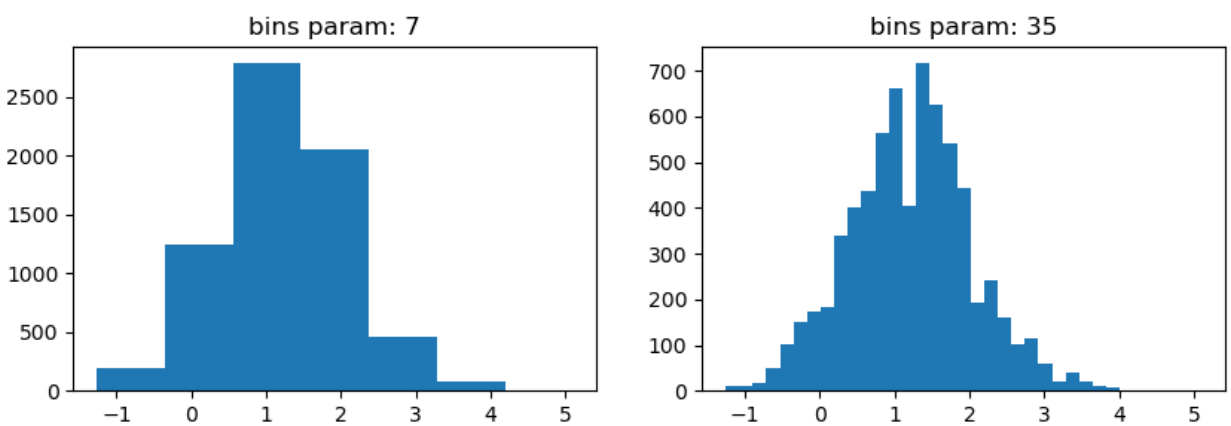
Histograms

```
quakes = pd.read_csv('earthquakes-1.csv')
plt.hist(quakes.query('magType == "ml"').mag)
plt.show()
```



Bin size matters

```
x = quakes.query('magType == "ml"').mag
fig, axes = plt.subplots(1, 2, figsize=(10, 3))
for ax, bins in zip(axes, [7, 35]):
    ax.hist(x, bins=bins)
    ax.set_title(f'bins param: {bins}')

plt.show()
```



Analysis

The histogram can be made by using plt.hist() and passing the data and the number of bins. The histogram shows the distribution of Facebook's stock prices. The choice of bin size can affect the appearance of the histogram, with smaller bins showing more detail and larger bins providing a smoother distribution.

Plot components

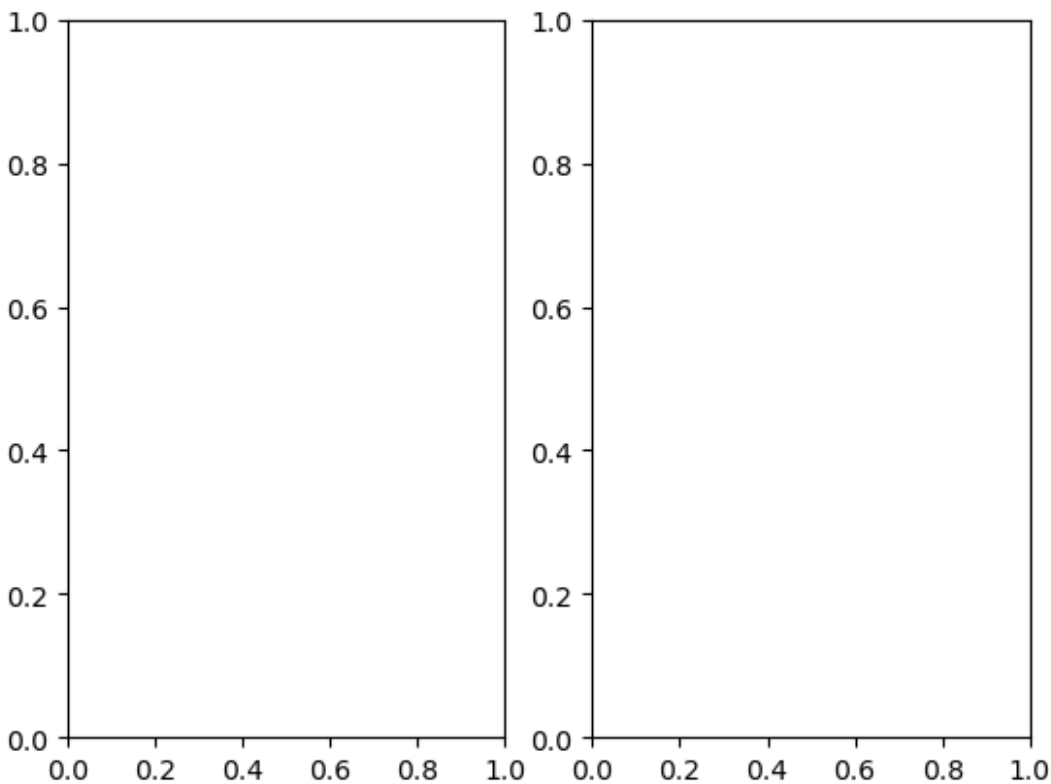
Figure

```
fig = plt.figure()
```

Axes

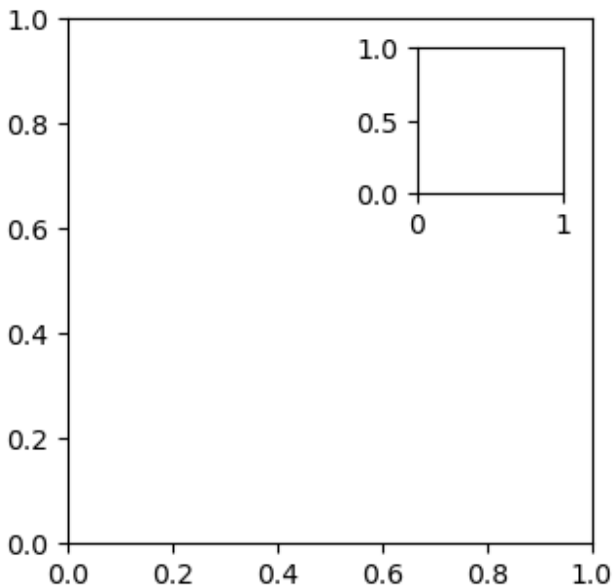
Creating subplots

```
fig, axes = plt.subplots(1, 2)

plt.show()
```




```
fig = plt.figure(figsize=(3, 3))
outside = fig.add_axes([0.1, 0.1, 0.9, 0.9])
```

```
inside = fig.add_axes([0.7, 0.7, 0.25, 0.25])
plt.show()
```

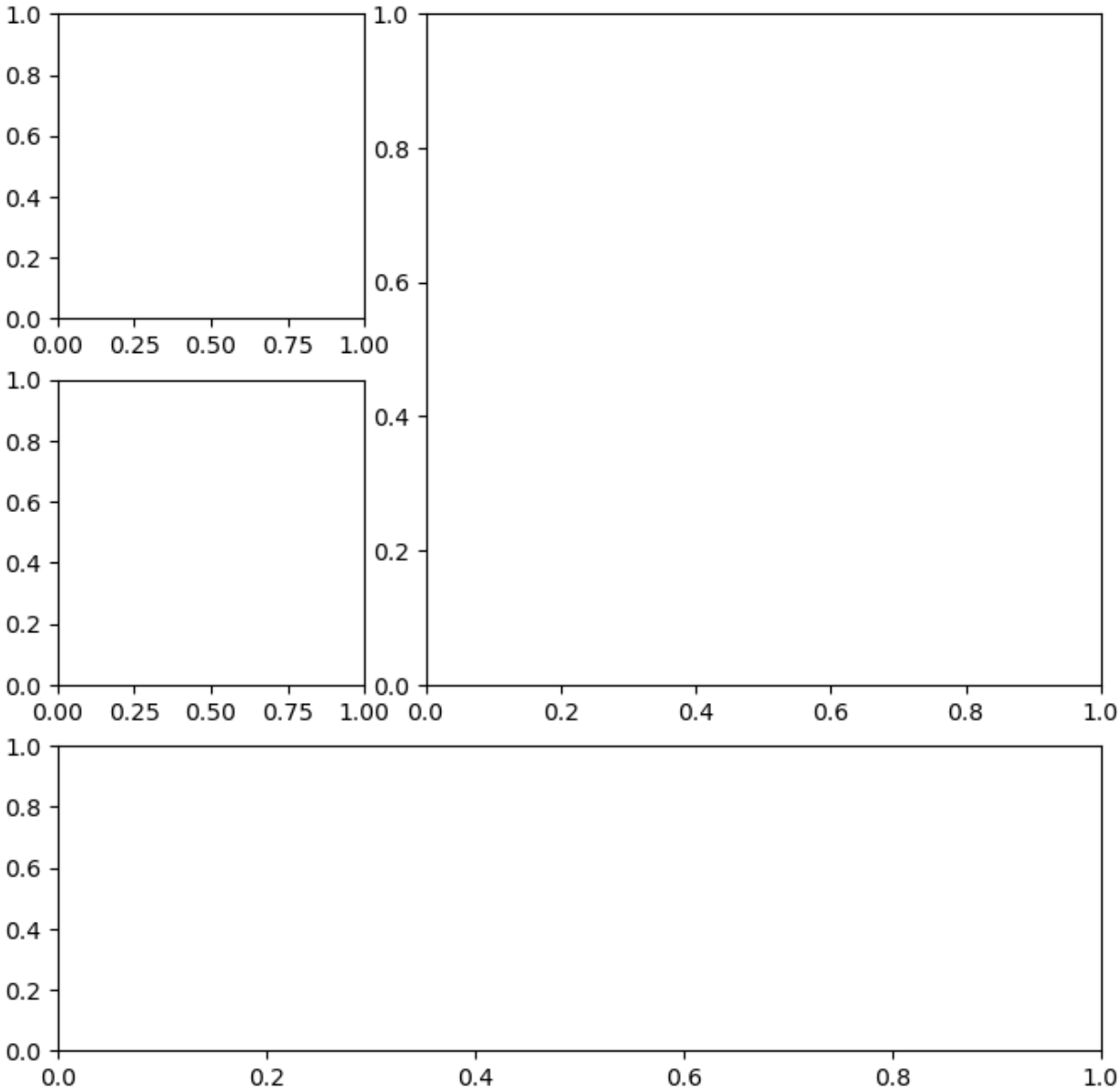


Analysis

The plot components include the figure, axes, and subplots. The figure is the overall container for the plot, while the axes are the individual plots within the figure. Subplots allow us to create multiple plots within a single figure, which can be useful for comparing different datasets or visualizing different aspects of the same dataset.

Creating Plot Layouts with gridspec

```
fig = plt.figure(figsize=(8, 8))
gs = fig.add_gridspec(3, 3)
top_left = fig.add_subplot(gs[0, 0])
mid_left = fig.add_subplot(gs[1, 0])
top_right = fig.add_subplot(gs[:2, 1:])
bottom = fig.add_subplot(gs[2,:])

plt.show()
```



Saving plots

```
fig.savefig('empty.png')
```

Cleaning up

```
plt.close('all')
```

Analysis

Creating subplots can be made by using plt.subplots() and passing the number of rows and columns. This allows us to create complex plot layouts and compare different datasets side by side.

Additional plotting options

Specifying figure size

```
fig = plt.figure(figsize=(10,4))

fig, axes = plt.subplots(1,2,figsize=(10,4))
```

rcParams

```
import random
import matplotlib as mpl

rcparams_list = list(mpl.rcParams.keys())
random.seed(20) # make this repeatable
random.shuffle(rcparams_list)
sorted(rcparams_list[:20])
```

```
['axes.edgecolor',
 'axes.formatter.limits',
 'boxplot.boxprops.color',
 'boxplot.flierprops.markersize',
 'boxplot.meanprops.markerfacecolor',
 'contour.corner_mask',
 'date.autoformatter.hour',
 'date.autoformatter.minute',
 'figure.subplot.top',
 'font.variant',
 'image.interpolation',
 'image.resample',
 'keymap.forward',
 'keymap.grid_minor',
 'pdf.use14corefonts',
 'xtick.minor.pad',
 'yaxis.labellocation',
 'ytick.labelleft',
 'ytick.major.size',
 'ytick.minor.visible']
```

```
mpl.rcParams['figure.figsize']
```

```
[6.4, 4.8]
```

```
mpl.rcParams['figure.figsize'] = (300, 10)
mpl.rcParams['figure.figsize']
```

```
[300.0, 10.0]
```

```
mpl.rcdefaults()
mpl.rcParams['figure.figsize']
```

```
[6.4, 4.8]
```

```
plt.rc('figure', figsize=(20, 20)) # change figsize default to (20, 20)
plt.rcdefaults() # reset the default
```

Analysis

Additional plotting options include specifying the figure size using plt.figure(figsize=(width, height)) and customizing the appearance of plots using rcParams. rcParams allows us to set default styles for all plots, such as colors, fonts, and line styles, which can help create a consistent look across multiple visualizations.

Plotting with Pandas

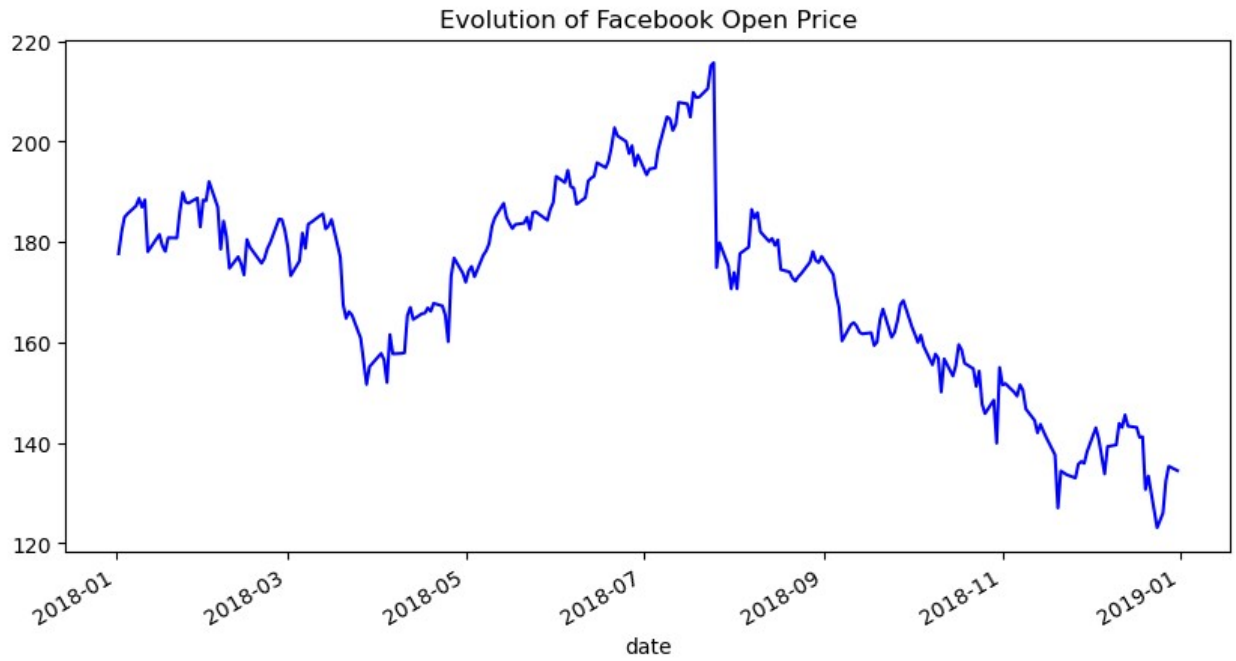
Setup

Evolution over time

```
fb.plot(
    kind='line',
    y='open',
    figsize=(10,5),
    style='b-',
    legend=False,
    title='Evolution of Facebook Open Price'
)
```

```
<Axes: title={'center': 'Evolution of Facebook Open Price'}, xlabel='date'>
```

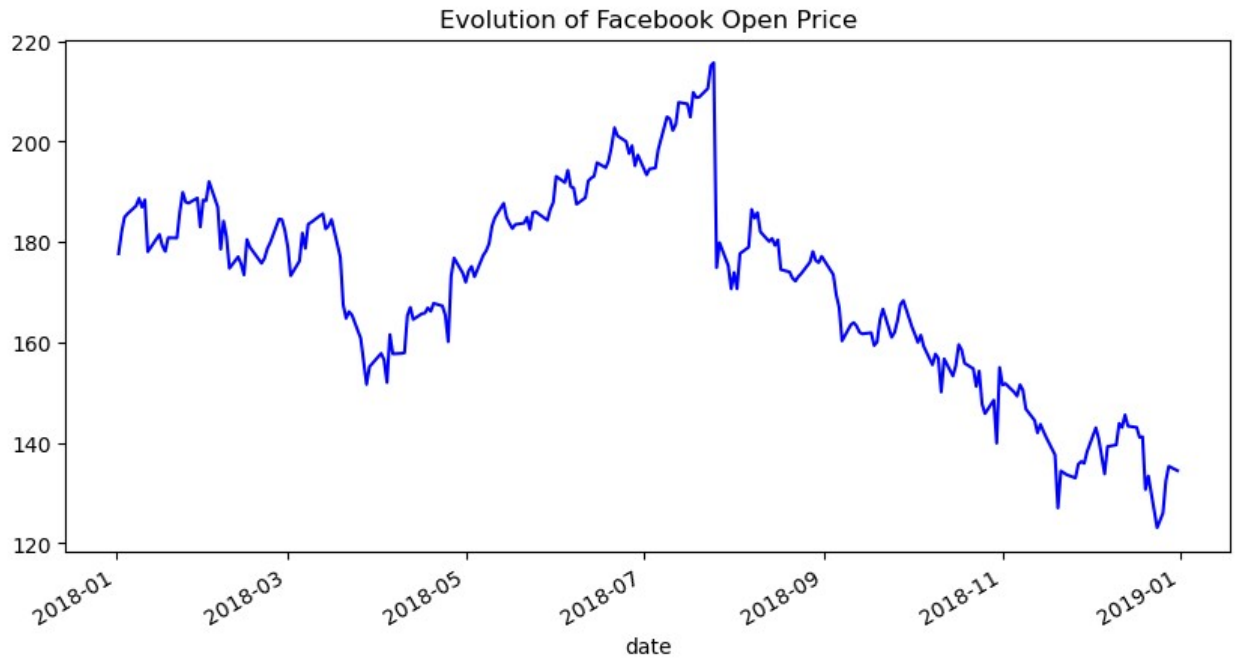
```
plt.show()
```




```
fb.plot(
    kind='line',
    y='open',
    figsize=(10,5),
    color='blue',
    linestyle='solid',
    legend=False,
    title='Evolution of Facebook Open Price'
)
```

```
<Axes: title={'center': 'Evolution of Facebook Open Price'},
xlabel='date'>
```

```
plt.show()
```



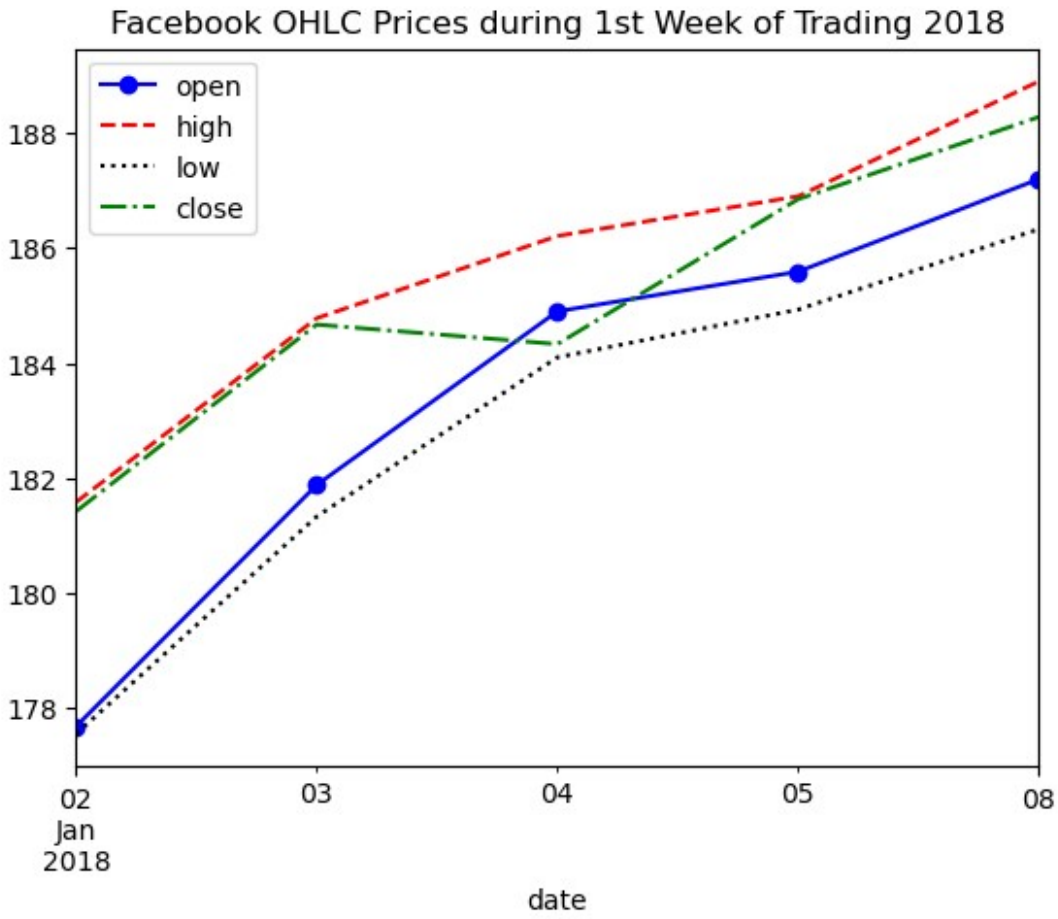
Analysis

The following line graphs show the evolution of Facebook's stock price over time. The graphs indicate an overall upward trend in Facebook's stock price throughout 2018 and a downward trend onwards. We used the plot function to create the line graphs and indicated the following parameters: x-axis, y-axis, title, labels, color, style, linestyle, and legend.

```
fb.iloc[:5,].plot(
    y=['open', 'high', 'low', 'close'],
    style=['b-o', 'r--', 'k:', 'g-.'],
    title='Facebook OHLC Prices during 1st Week of Trading 2018'
)
```

```
<Axes: title={'center': 'Facebook OHLC Prices during 1st Week of
Trading 2018'}, xlabel='date'>
```

```
plt.show()
```

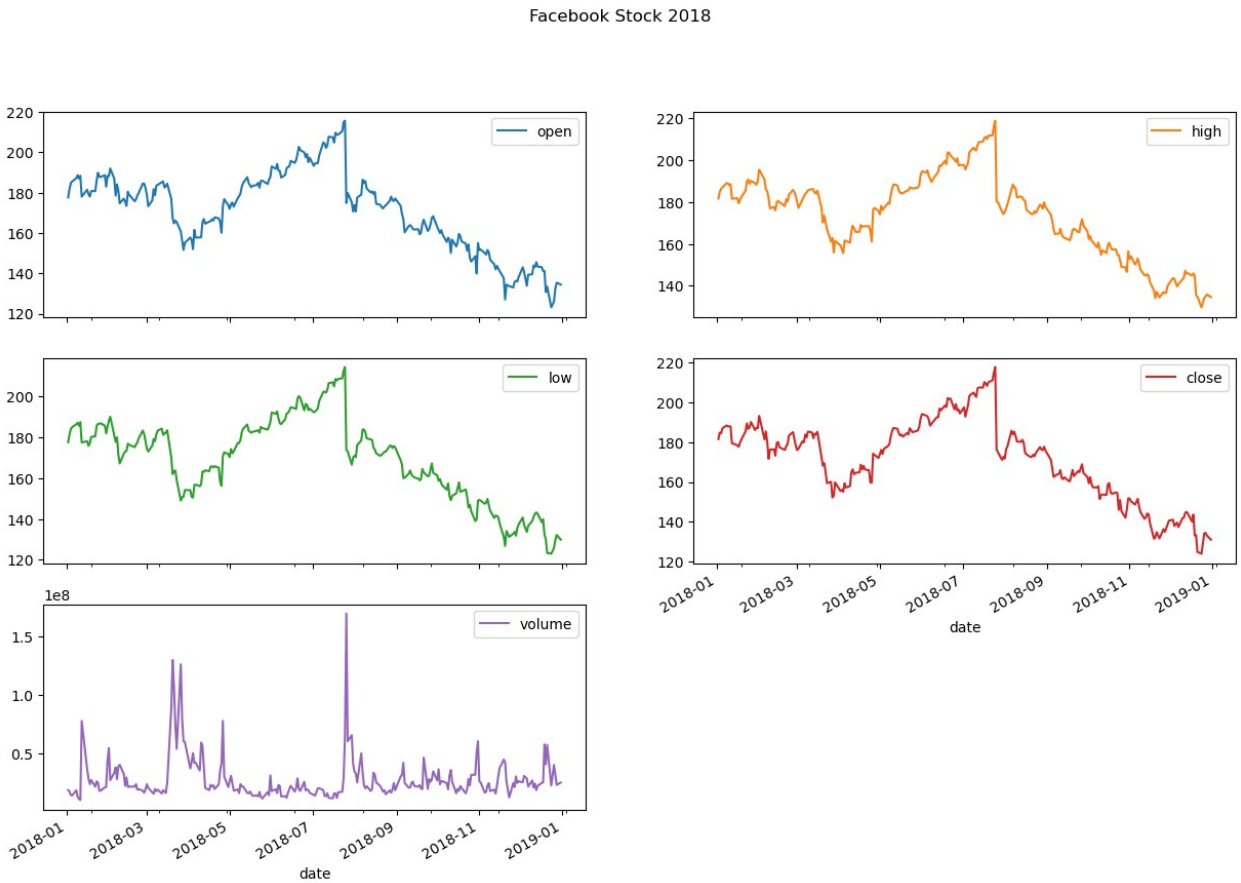


Analysis

The following line graph shows a more complex visualization of Facebook's stock price over time, with additional parameters such as markers and gridlines. The graph indicates the trends on the OHLC of the stock, with different colors and markers for each variable. The gridlines help to improve readability and make it easier to compare the different variables.

Creating subplots

```
fb.plot(
    kind='line',
    subplots=True,
    layout=(3,2),
    figsize=(15,10),
    title='Facebook Stock 2018'
)

array([[<Axes: xlabel='date'>, <Axes: xlabel='date'>],
       [<Axes: xlabel='date'>, <Axes: xlabel='date'>],
       [<Axes: xlabel='date'>, <Axes: xlabel='date'>]], dtype=object)

plt.show()
```



Analysis

The creation of subplots help us see through the Facebook stock price data in a more comprehensive way. The first subplot shows the line graph of the stock price over time, while the second subplot shows a scatter plot of the high and low stock prices. This allows us to see both the overall trend and the correlation between the high and low prices in one figure.

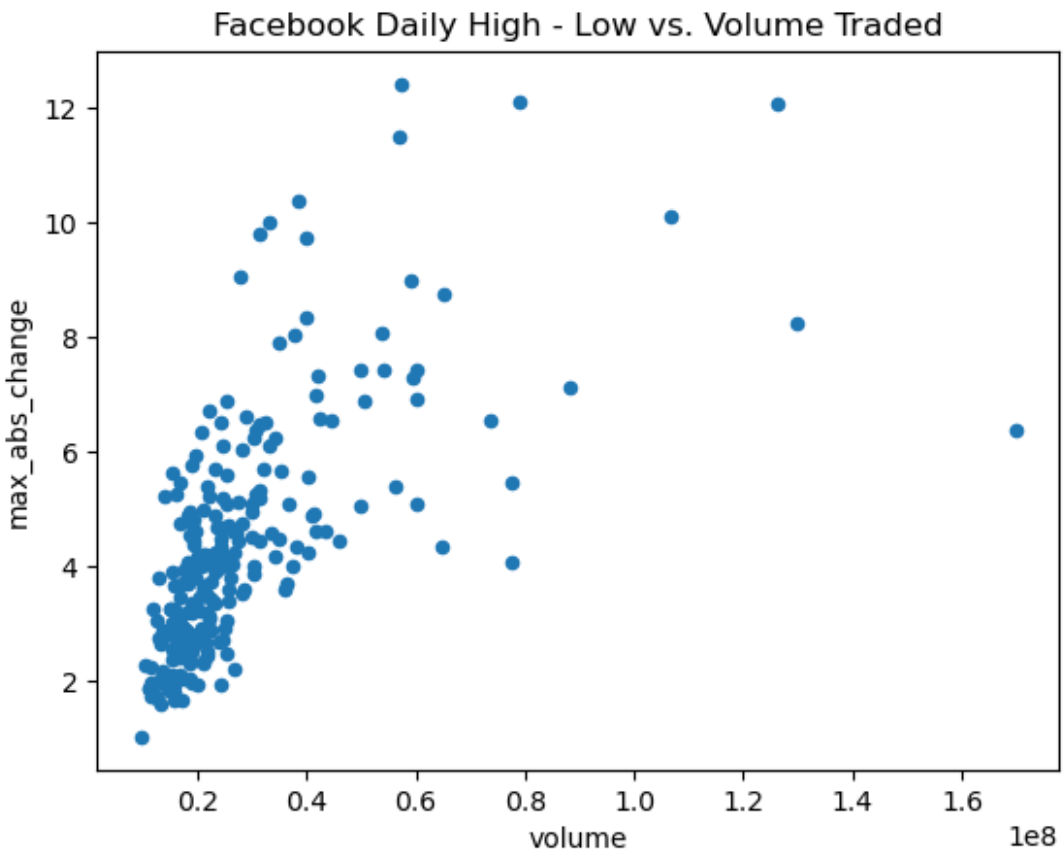
Visualizing relationship between variables

Scatter plots

```
fb.assign(
    max_abs_change = fb.high - fb.low
).plot(
    kind='scatter', x='volume', y='max_abs_change',
    title='Facebook Daily High - Low vs. Volume Traded'
)
```

```
<Axes: title={'center': 'Facebook Daily High - Low vs. Volume Traded'}, xlabel='volume', ylabel='max_abs_change'>
```

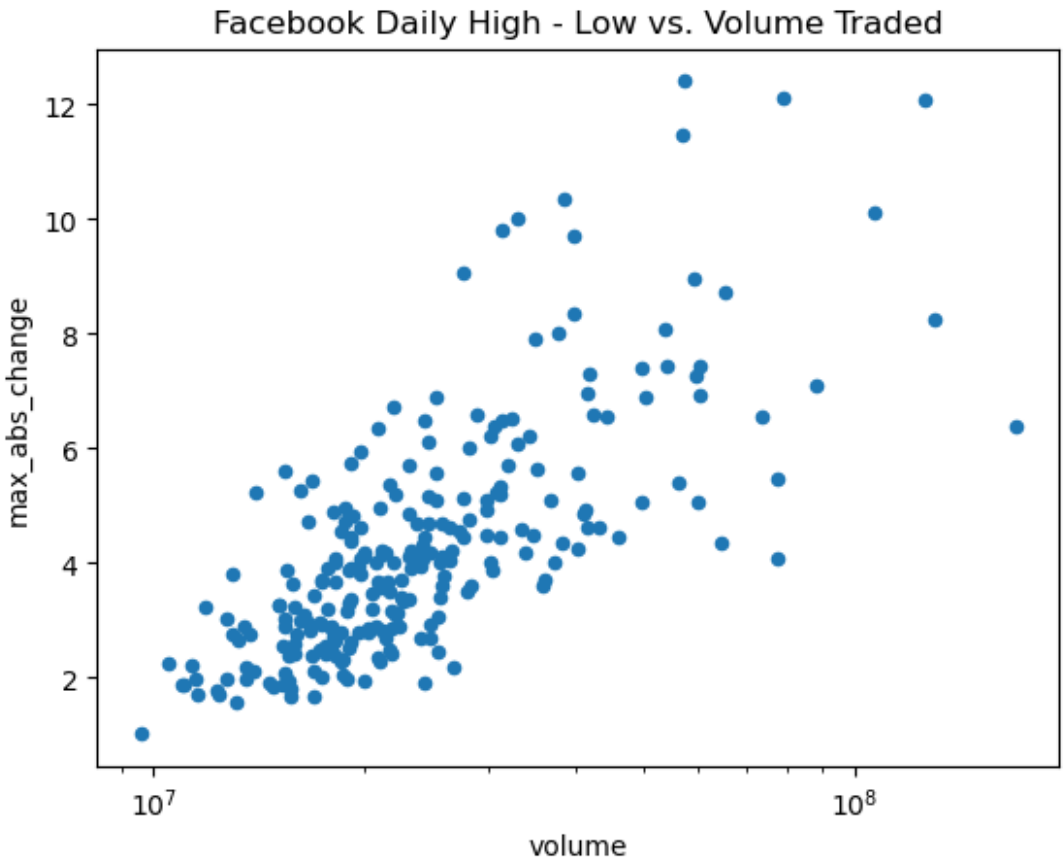
```
plt.show()
```




```
fb.assign(
    max_abs_change = fb.high - fb.low
).plot(
    kind='scatter', x='volume', y='max_abs_change',
    title='Facebook Daily High - Low vs. Volume Traded',
    logx=True
)
```

```
<Axes: title={'center': 'Facebook Daily High - Low vs. Volume
Traded'}, xlabel='volume', ylabel='max_abs_change'>
```

```
plt.show()
```



Analysis

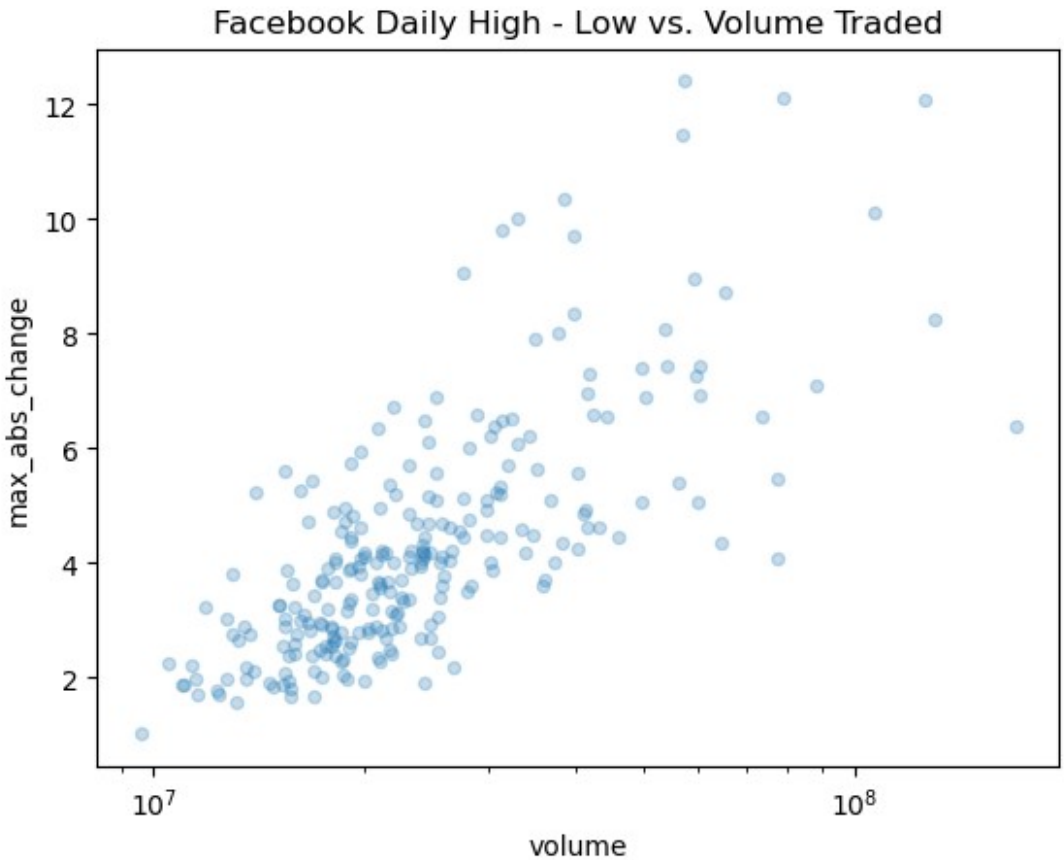
To visualize the relationship between variables, we can use scatter plots. The scatter plot of the high and low stock prices of Facebook shows a positive correlation, indicating that as the high price increases, the low price also tends to increase. We used the apply function to create a new column for the difference between the high and low prices, and then created a scatter plot to visualize this relationship.

Adding Transparency to Plots with alpha

```
fb.assign(
    max_abs_change = fb.high - fb.low
).plot(
    kind='scatter', x='volume', y='max_abs_change',
    title='Facebook Daily High - Low vs. Volume Traded',
    logx=True, alpha=0.25
)
```

```
<Axes: title={'center': 'Facebook Daily High - Low vs. Volume
Traded'}, xlabel='volume', ylabel='max_abs_change'>
```

```
plt.show()
```



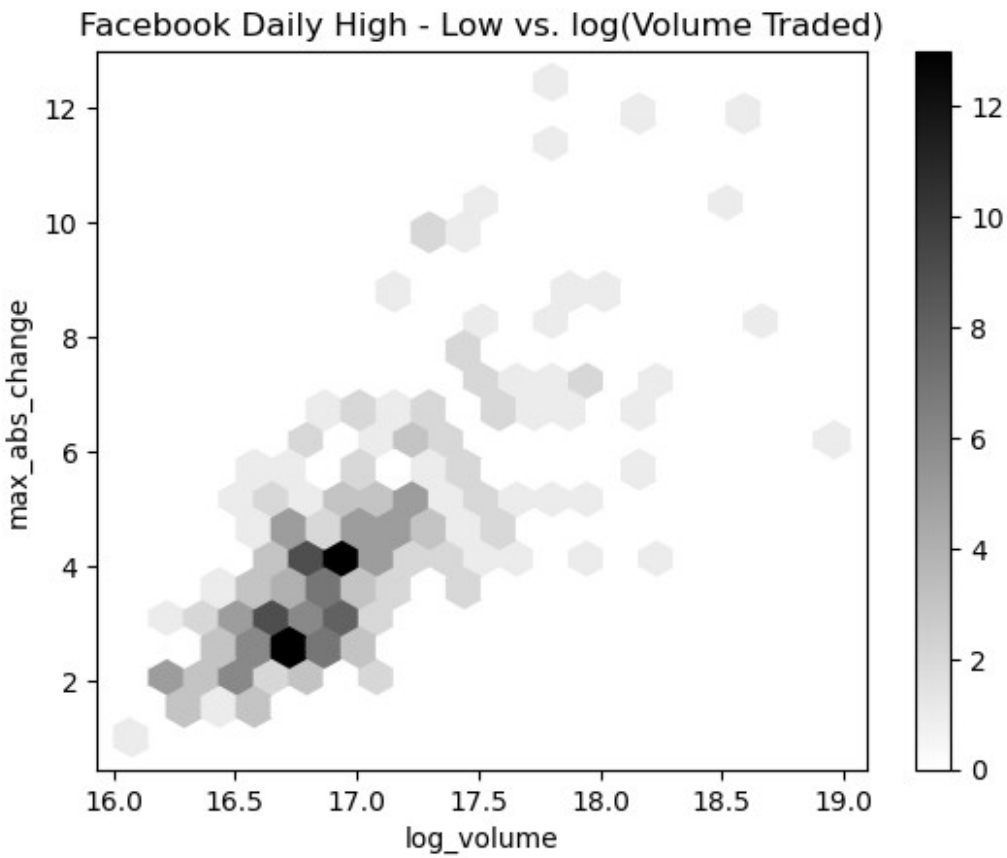
Analysis

Adding transparency to plots can be done by using the alpha parameter in plotting functions. This can help to reduce clutter and make it easier to see overlapping points in scatter plots or multiple lines in line graphs.

Hexbins

```
fb.assign(
log_volume=np.log(fb.volume),
max_abs_change=fb.high - fb.low
).plot(
    kind='hexbin',
    x='log_volume',
    y='max_abs_change',
    title='Facebook Daily High - Low vs. log(Volume Traded)',
    colormap='gray_r',
    gridsize=20,
    sharex=False # we have to pass this to see the x-axis due to a bug
in this version of pandas
)
```

```
<Axes: title={'center': 'Facebook Daily High - Low vs. log(Volume Traded)'}, xlabel='log_volume', ylabel='max_abs_change'>

plt.show()
```



Analysis

Hexbin plots can be created using the hexbin function in Matplotlib, which creates a hexagonal binning plot. This type of plot is useful for visualizing the density of points in a scatter plot, especially when there are a large number of points that may overlap.

Visualizing Correlations with Heatmaps

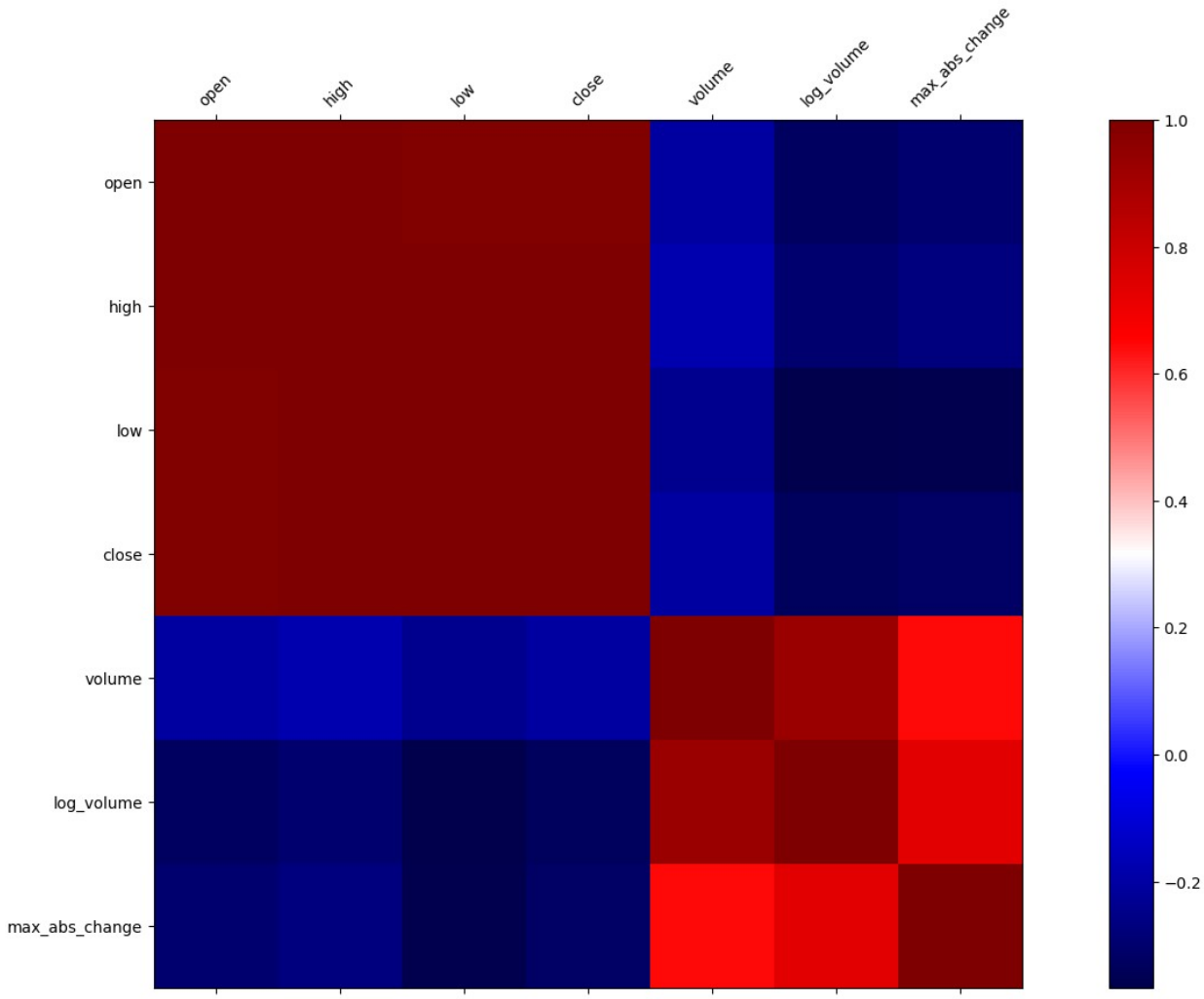
```
fig, ax = plt.subplots(figsize=(20,10))

fb_corr = fb.assign(
    log_volume = np.log(fb.volume),
    max_abs_change = fb.high - fb.low
).corr()

im = ax.matshow(fb_corr, cmap='seismic')
fig.colorbar(im)
```

```
labels=[col.lower() for col in fb_corr.columns]
ax.set_xticklabels([''] + labels, rotation=45)
ax.set_yticklabels([''] + labels)
```

```
C:\Users\Aldous\AppData\Local\Temp\ipykernel_12492\2991456283.py:12:
UserWarning: set_ticklabels() should only be used with a fixed number
of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels([''] + labels, rotation=45)
C:\Users\Aldous\AppData\Local\Temp\ipykernel_12492\2991456283.py:13:
UserWarning: set_ticklabels() should only be used with a fixed number
of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_yticklabels([''] + labels)
```

```
[Text(0, -1.0, ''),
 Text(0, 0.0, 'open'),
 Text(0, 1.0, 'high'),
 Text(0, 2.0, 'low'),
 Text(0, 3.0, 'close'),
 Text(0, 4.0, 'volume'),
 Text(0, 5.0, 'log_volume'),
 Text(0, 6.0, 'max_abs_change'),
 Text(0, 7.0, '')]
```

```
plt.show()
```




```
fb_corr.loc['max_abs_change', ['volume', 'log_volume']]
```

```
volume        0.642027
log_volume    0.731542
Name: max_abs_change, dtype: float64
```

Analysis

Heatmaps are useful for visualizing the correlation between multiple variables in a dataset, with colors representing the strength of the correlation.

Visualizing distributions

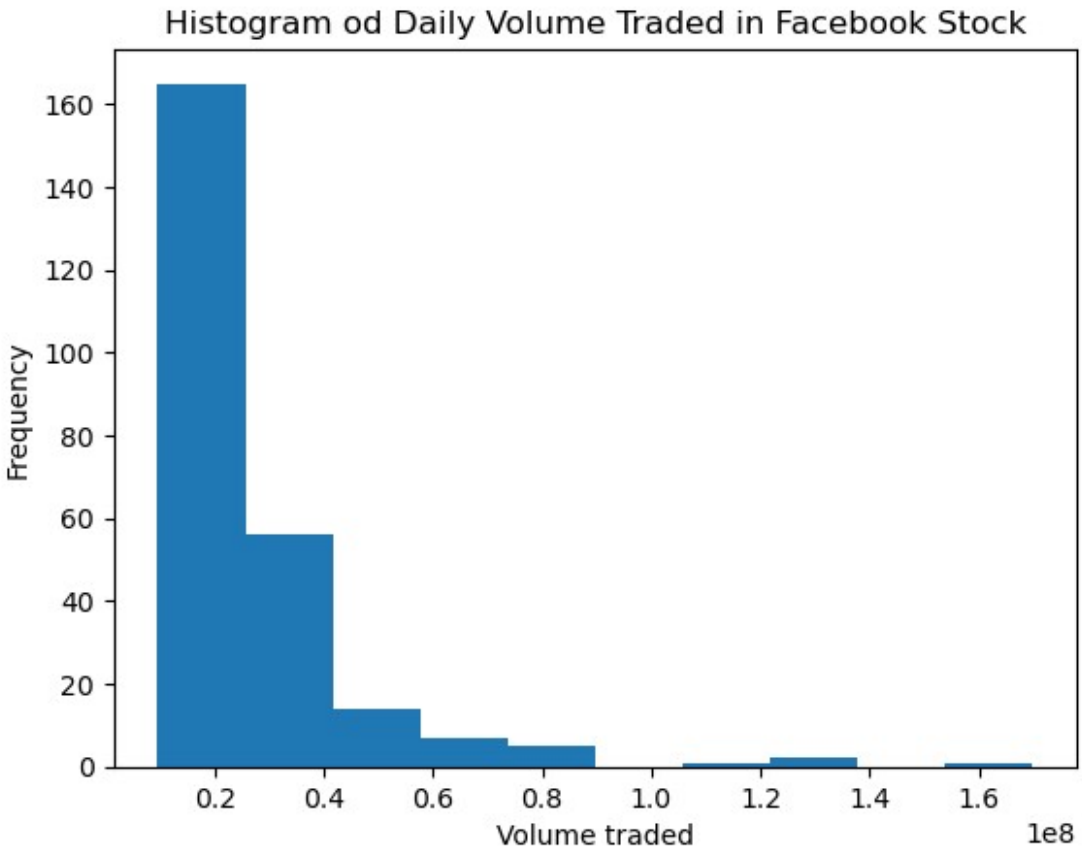
Histograms

```
fb.volume.plot(
    kind='hist',
    title='Histogram od Daily Volume Traded in Facebook Stock'
```

```
)
plt.xlabel('Volume traded')
```

```
Text(0.5, 0, 'Volume traded')
```

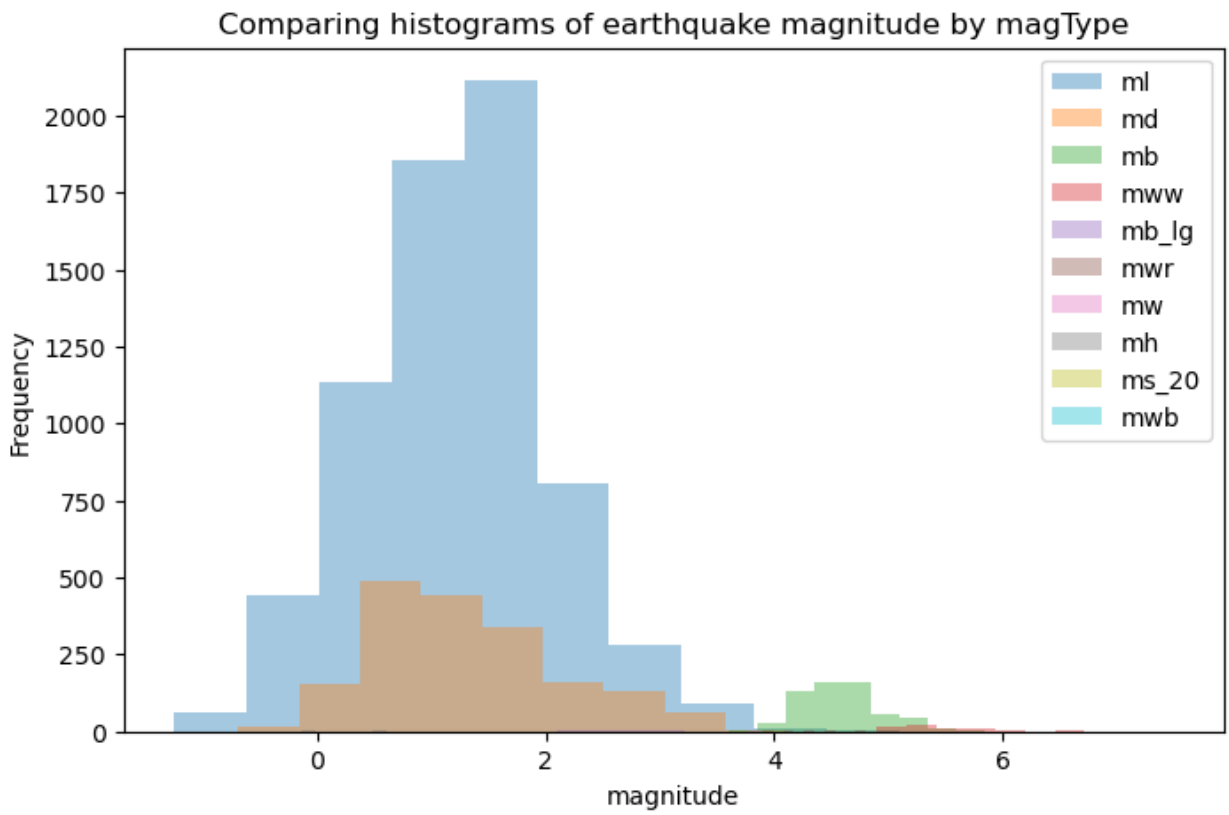
```
plt.show()
```




```
fig, axes = plt.subplots(figsize=(8, 5))
for magtype in quakes.magType.unique():
    data = quakes.query(f'magType == "{magtype}"').mag
    if not data.empty:
        data.plot(
            kind='hist', ax=axes, alpha=0.4,
            label=magtype, legend=True,
            title='Comparing histograms of earthquake magnitude by
magType'
        )
plt.xlabel('magnitude')
```

```
Text(0.5, 0, 'magnitude')
```

```
plt.show()
```



Analysis

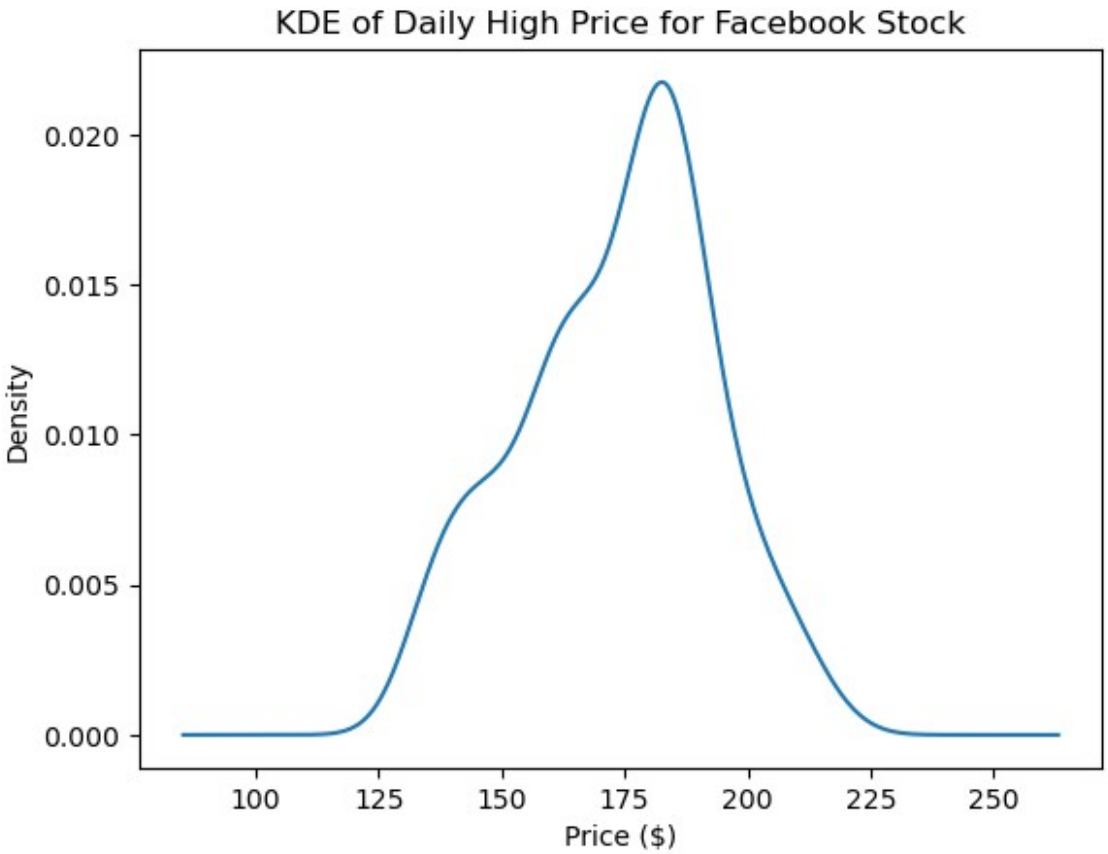
Comparison of histograms can be made by adding all histograms in one figure and using the alpha parameter to differentiate them. This allows us to compare the distributions of different variables in a single plot, making it easier to identify similarities and differences between them.

Kernel Density Estimation (KDE)

```
fb.high.plot(
    kind='kde',
    title='KDE of Daily High Price for Facebook Stock'
)
plt.xlabel('Price ($)')
```

```
Text(0.5, 0, 'Price ($)')
```

```
plt.show()
```

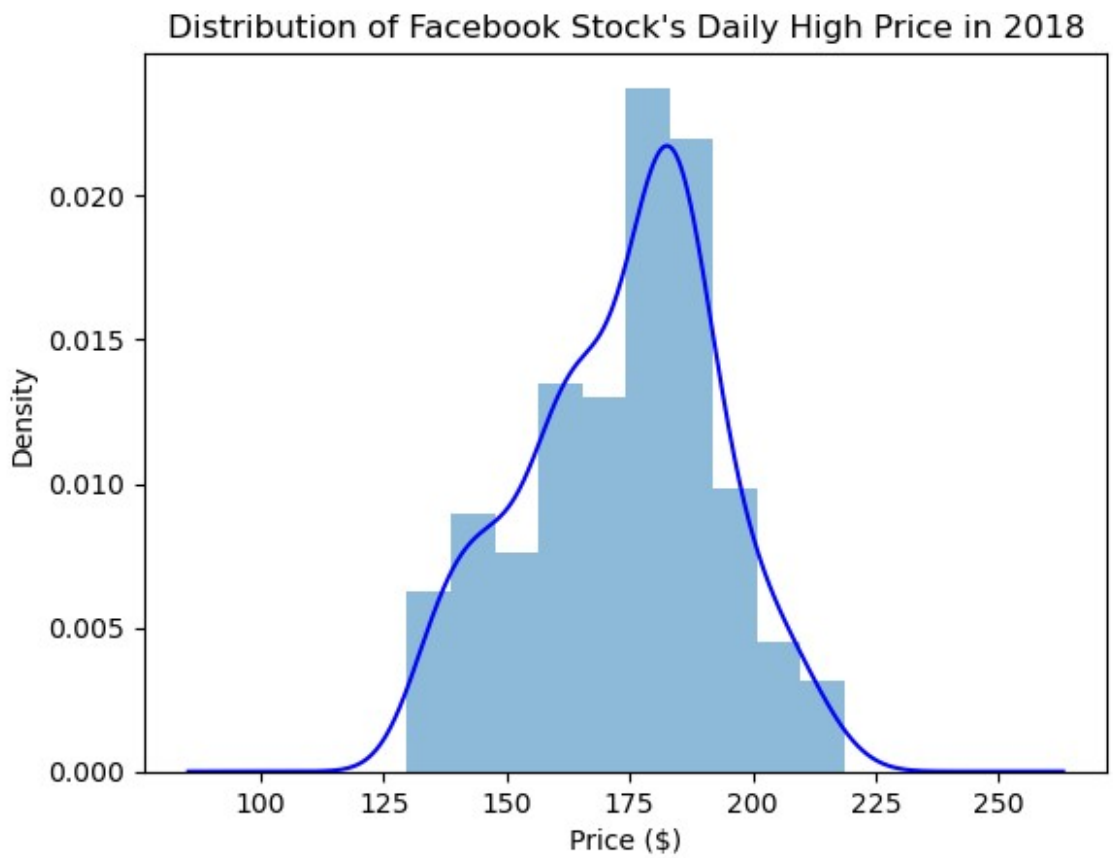


Adding to the result of plot()

```
ax = fb.high.plot(kind='hist', density=True, alpha=0.5)
fb.high.plot(
    ax=ax, kind='kde', color='blue',
    title='Distribution of Facebook Stock\'s Daily High Price in 2018'
)
plt.xlabel('Price ($)')
```

```
Text(0.5, 0, 'Price ($)')
```

```
plt.show()
```



Analysis

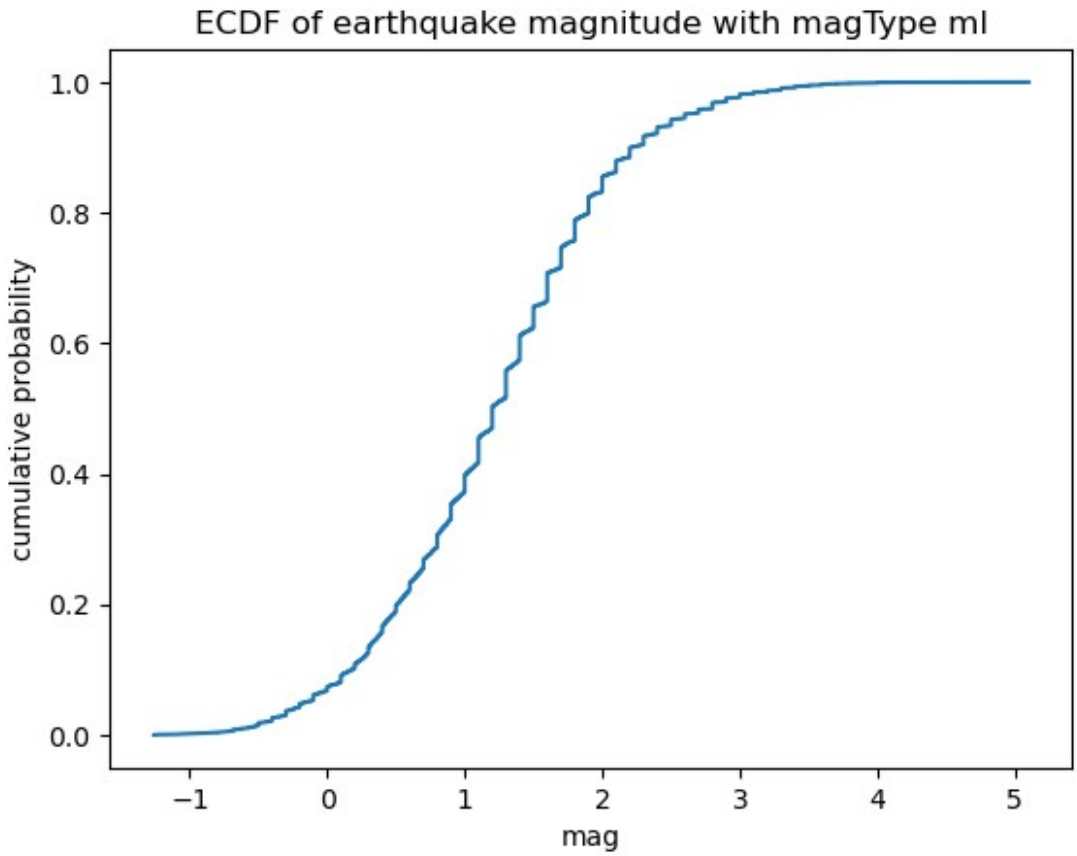
The KDE plot can be made by using the kde parameter in the plot function. This allows us to visualize the distribution of data in a smooth curve, which can be helpful for identifying patterns and trends in the data. We can add this with a histogram to see both the histogram and the KDE curve in one plot, providing a more comprehensive view of the data distribution.

Plotting the ECDF

```
from statsmodels.distributions.empirical_distribution import ECDF
ecdf = ECDF(quakes.query('magType == "ml"').mag)
plt.plot(ecdf.x, ecdf.y)

# axis labels (we will cover this in chapter 6)
plt.xlabel('mag') # add x-axis label
plt.ylabel('cumulative probability') # add y-axis label

# add title (we will cover this in chapter 6)
plt.title('ECDF of earthquake magnitude with magType ml')
```

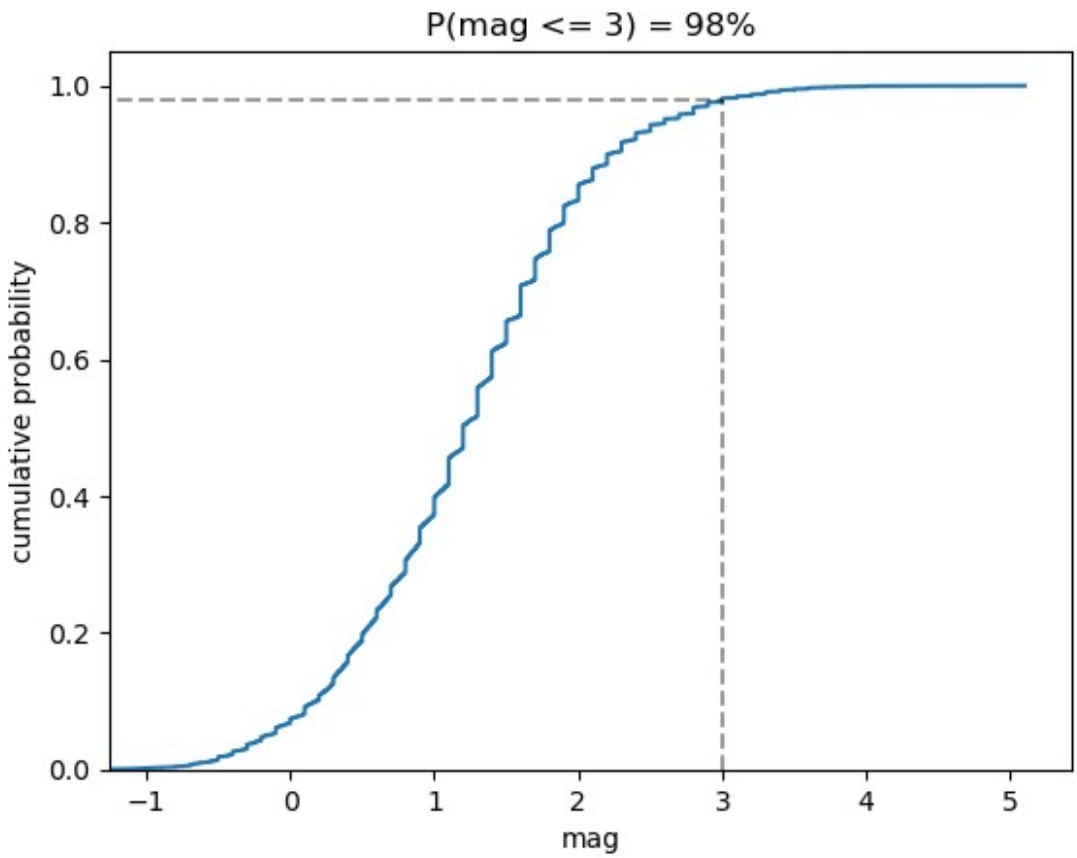
```
Text(0.5, 1.0, 'ECDF of earthquake magnitude with magType ml')
```

```
plt.show()
```




```
from statsmodels.distributions.empirical_distribution import ECDF
ecdf = ECDF(quakes.query('magType == "ml"').mag)
plt.plot(ecdf.x, ecdf.y)

# formatting below will all be covered in chapter 6
# axis labels
plt.xlabel('mag') # add x-axis label
plt.ylabel('cumulative probability') # add y-axis label

# add reference lines for interpreting the ECDF for mag <= 3
plt.plot(
[3, 3], [0, .98], 'k--',
[-1.5, 3], [0.98, 0.98], 'k--', alpha=0.4
)

# set axis ranges
plt.ylim(0, None)
plt.xlim(-1.25, None)

# add a title
plt.title('P(mag <= 3) = 98%')

Text(0.5, 1.0, 'P(mag <= 3) = 98%')
```

```
plt.show()
```



Analysis

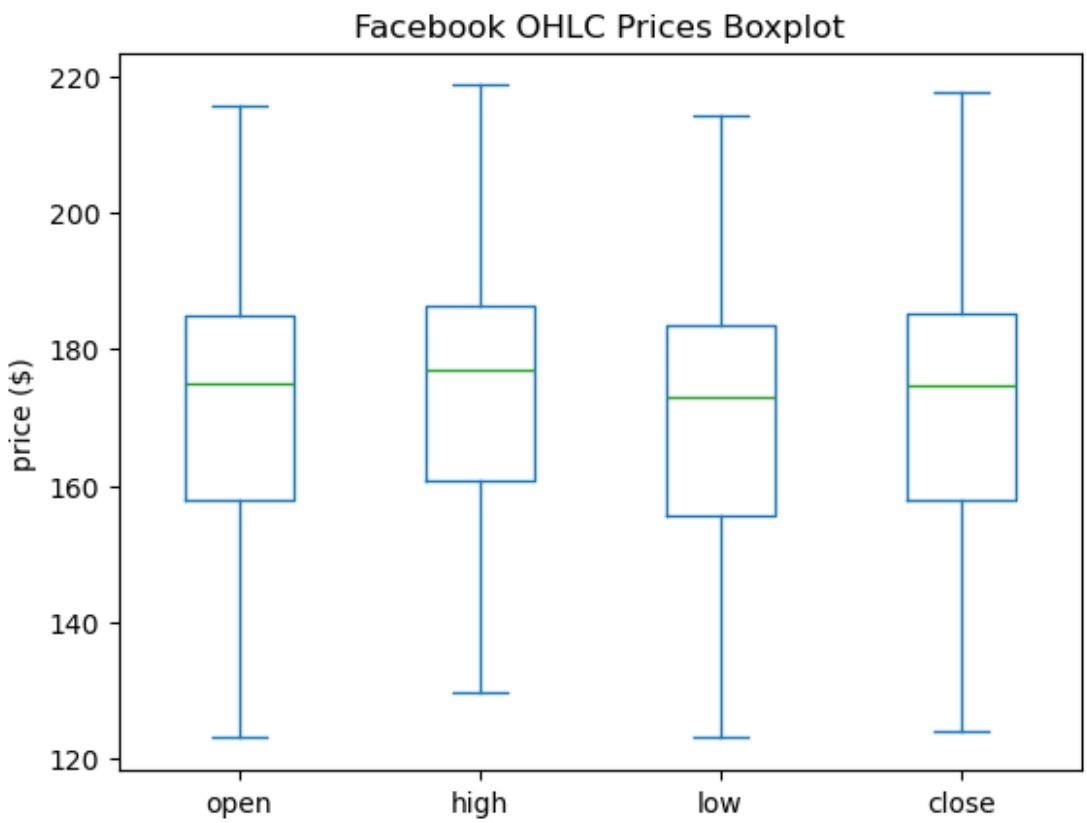
The ECDF plot can be made by using the ecdf parameter in the plot function. This allows us to visualize the cumulative distribution of data, which can be helpful for understanding the proportion of data points that fall below a certain value. The ECDF plot provides a way to compare distributions and identify differences between datasets.

Box plots

```
fb.iloc[:,:4].plot(kind='box', title='Facebook OHLC Prices Boxplot')
plt.ylabel('price ($)') # label the x-axis (discussed in chapter 6)
```

```
Text(0, 0.5, 'price ($)')
```

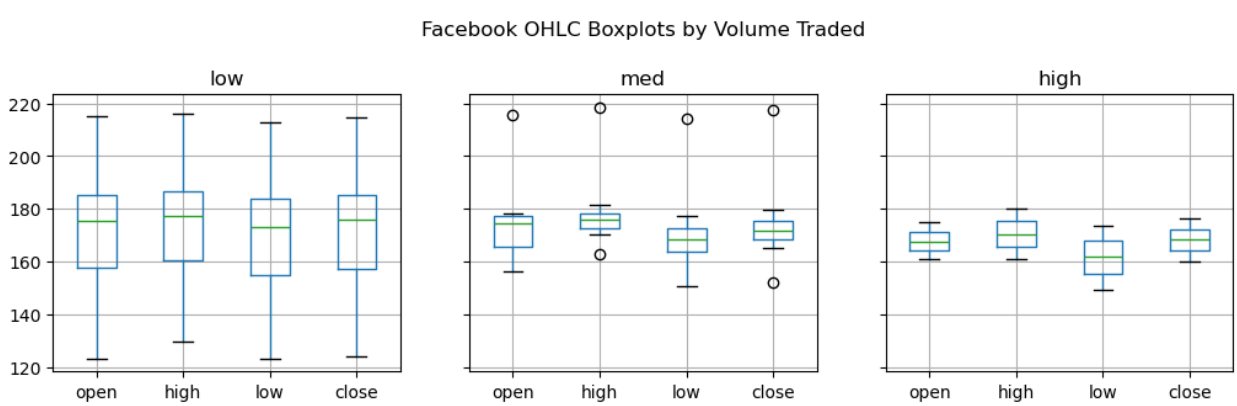
```
plt.show()
```




```
fb.assign(
volume_bin=pd.cut(fb.volume, 3, labels=['low', 'med', 'high'])
).groupby('volume_bin').boxplot(
    column=['open', 'high', 'low', 'close'],
    layout=(1, 3), figsize=(12, 3)
)
plt.suptitle('Facebook OHLC Boxplots by Volume Traded', y=1.1)
```

```
Text(0.5, 1.1, 'Facebook OHLC Boxplots by Volume Traded')
```

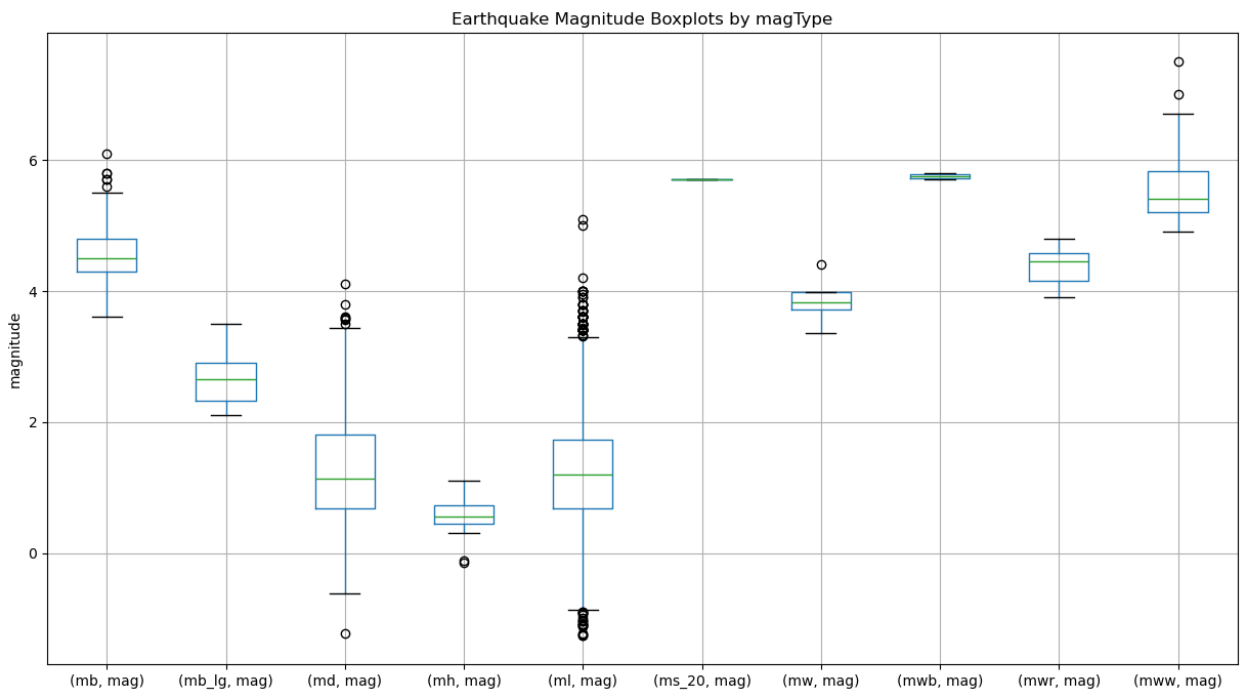
```
plt.show()
```



```
quakes[['mag', 'magType']].groupby('magType').boxplot(
    figsize=(15, 8), subplots=False
)
plt.title('Earthquake Magnitude Boxplots by magType')
plt.ylabel('magnitude') # label the y-axis (discussed in chapter 6)
```

```
Text(0, 0.5, 'magnitude')
```

```
plt.show()
```



Analysis

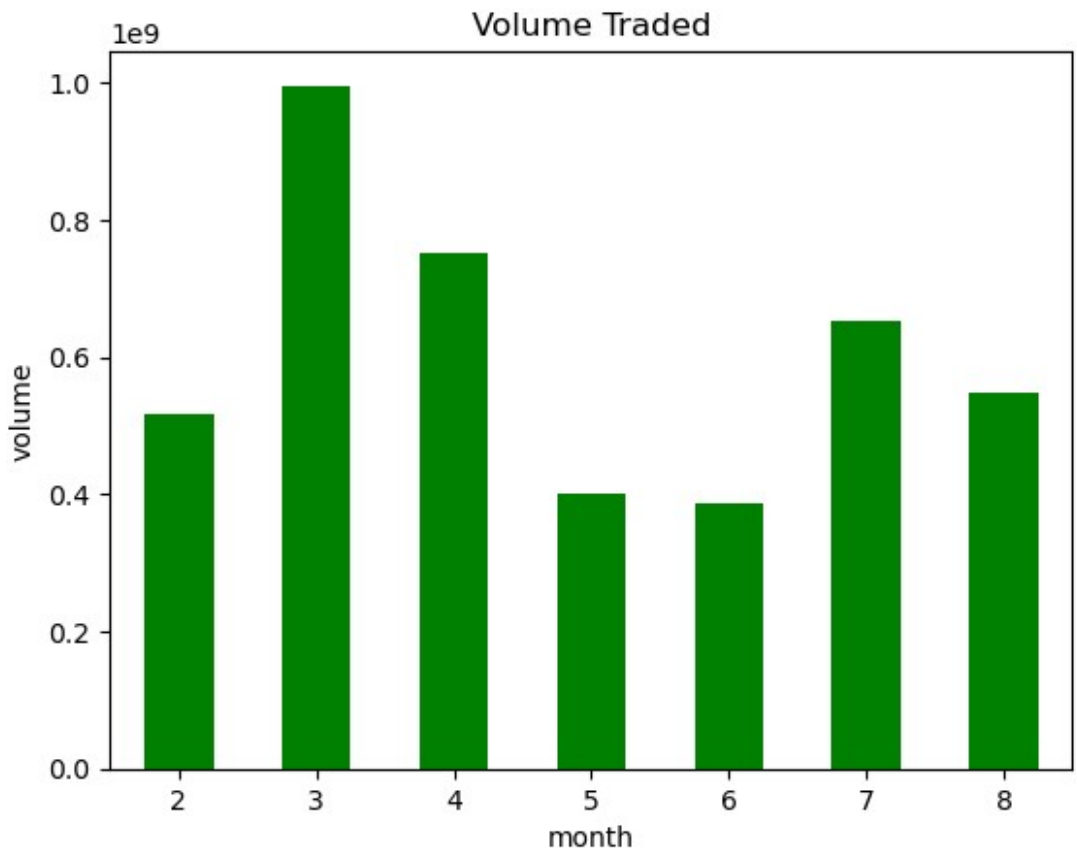
The box plot can be made by using the boxplot function in Matplotlib. This type of plot is useful for visualizing the distribution of data and identifying outliers. The box plot shows the median, quartiles, and potential outliers in the dataset, providing insights into the spread and central tendency of the data.

Counts and frequencies

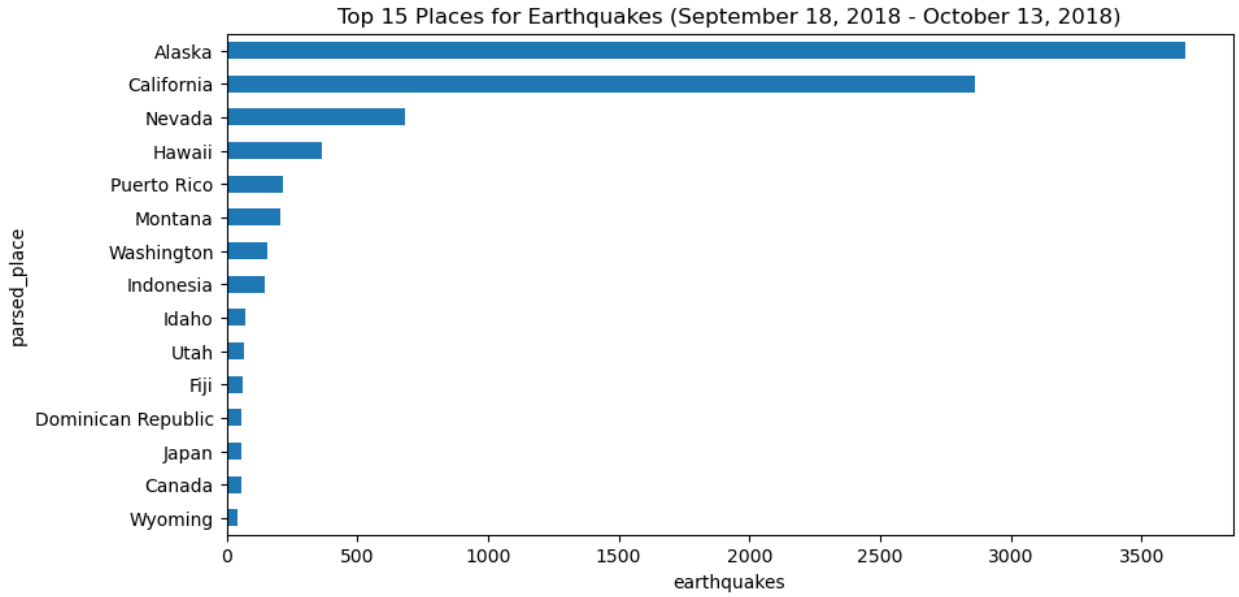
Bar charts

```
fb['2018-02':'2018-08'].assign(
    month=lambda x: x.index.month
).groupby('month').sum().volume.plot.bar(
    color='green', rot=0, title='Volume Traded'
)
plt.ylabel('volume') # label the y-axis (discussed in chapter 6)
```

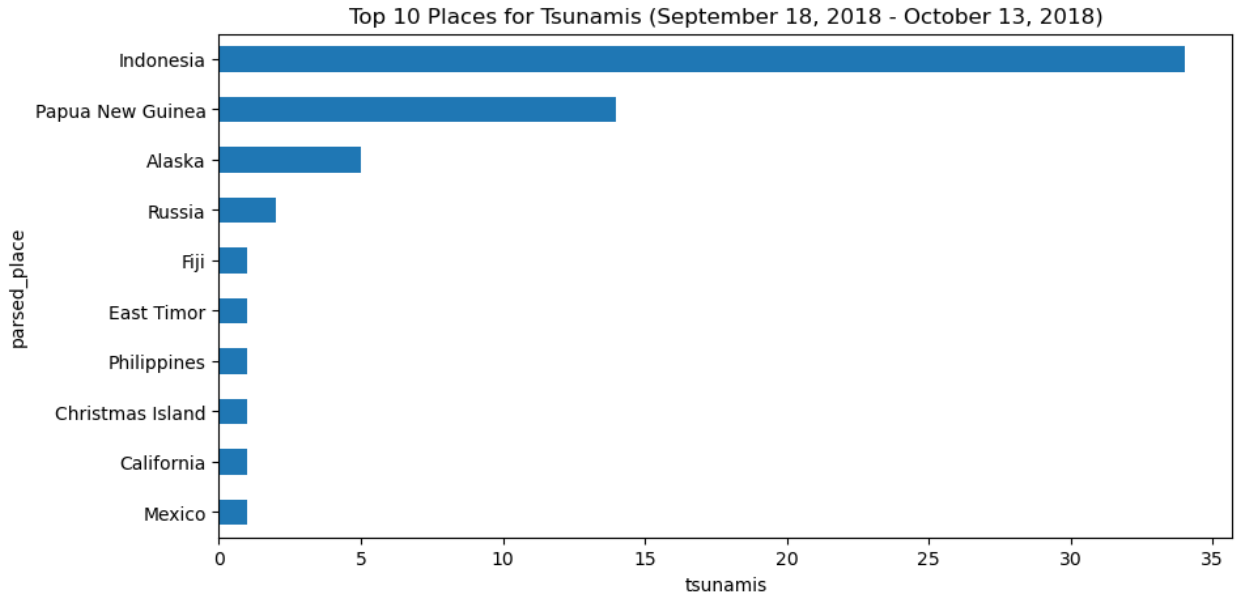
```
Text(0, 0.5, 'volume')

plt.show()
```




```
quakes.parsed_place.value_counts().iloc[14::-1,].plot(
    kind='barh', figsize=(10, 5),
    title='Top 15 Places for Earthquakes '\
    '(September 18, 2018 - October 13, 2018)'
)
plt.xlabel('earthquakes') # label the x-axis (discussed in chapter 6)

Text(0.5, 0, 'earthquakes')

plt.show()
```



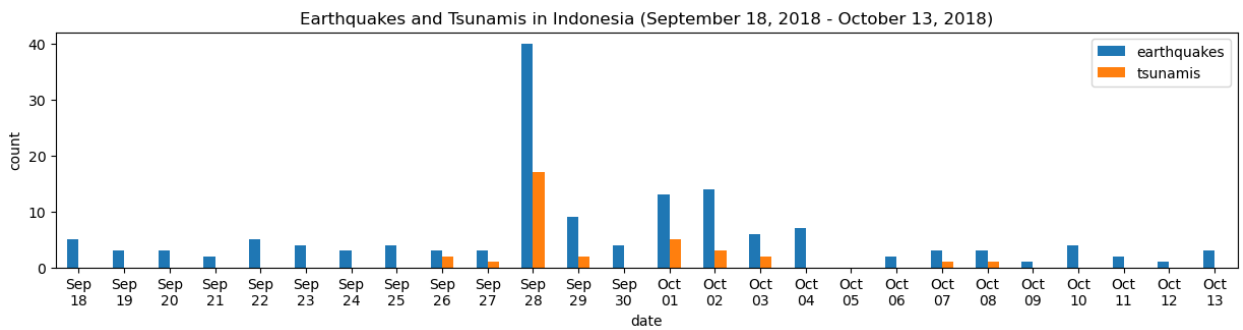

```
quakes.groupby('parsed_place').tsunami.sum().sort_values().iloc[-
10::,].plot(
    kind='barh', figsize=(10, 5),
    title='Top 10 Places for Tsunamis '\
    '(September 18, 2018 - October 13, 2018)'
)
plt.xlabel('tsunamis') # label the x-axis (discussed in chapter 6)

Text(0.5, 0, 'tsunamis')

plt.show()
```



```
indonesia_quakes = quakes.query('parsed_place == "Indonesia"').assign(
    time=lambda x: pd.to_datetime(x.time, unit='ms'),
    earthquake=1
).set_index('time').resample('1D').sum()

indonesia_quakes.index = indonesia_quakes.index.strftime('%b\n%d')

indonesia_quakes.plot(
    y=['earthquake', 'tsunami'], kind='bar', figsize=(15, 3), rot=0,
    label=['earthquakes', 'tsunamis'],
    title='Earthquakes and Tsunamis in Indonesia '\
    '(September 18, 2018 - October 13, 2018)'
)
# label the axes (discussed in chapter 6)
plt.xlabel('date')
plt.ylabel('count')
```

```
Text(0, 0.5, 'count')
```

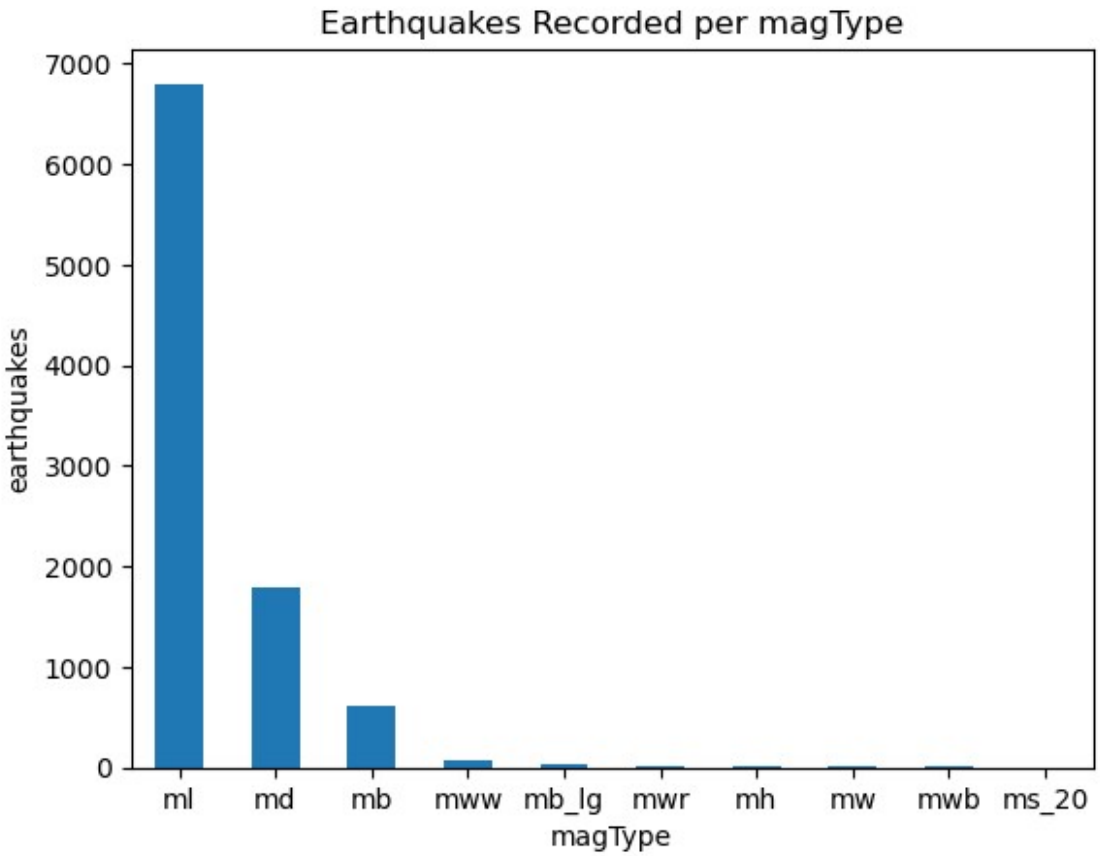
```
plt.show()
```




```
quakes.magType.value_counts().plot(
    kind='bar', title='Earthquakes Recorded per magType', rot=0
)
# label the axes (discussed in chapter 6)
plt.xlabel('magType')
plt.ylabel('earthquakes')
```

```
Text(0, 0.5, 'earthquakes')
```

```
plt.show()
```

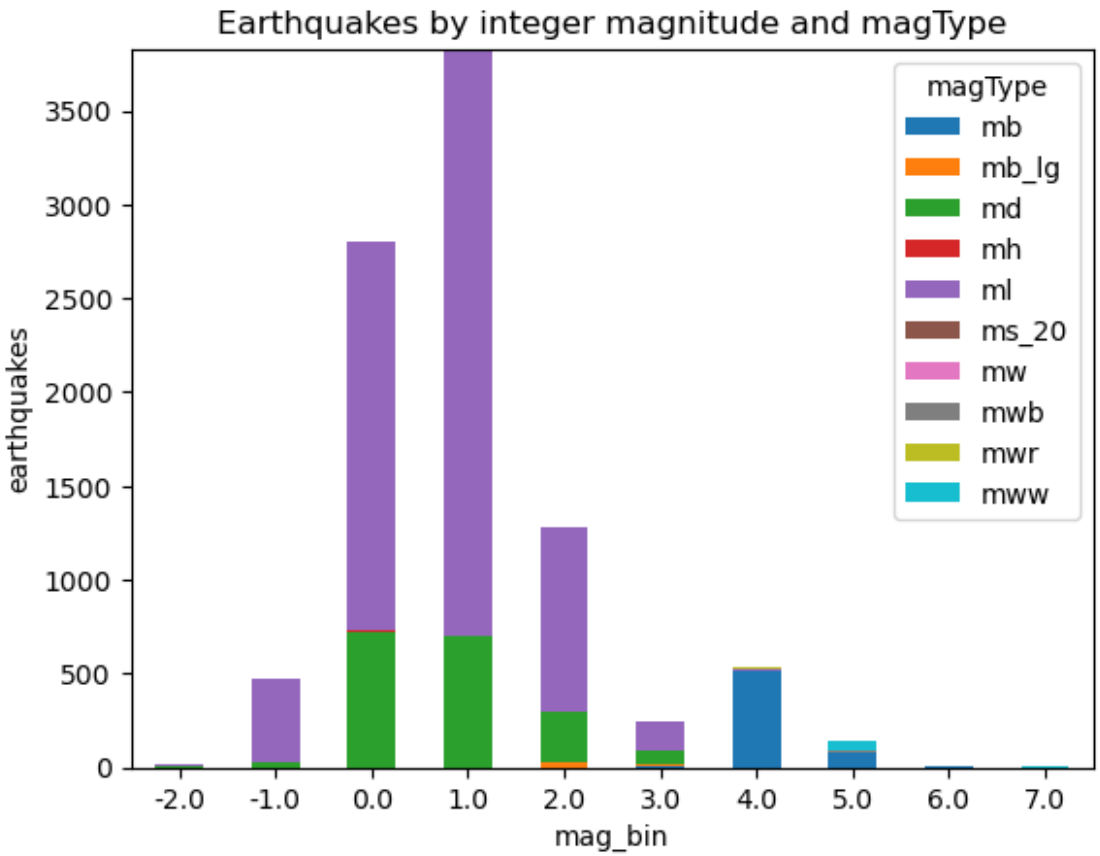


Stacked bar chart

```
pivot = quakes.assign(
mag_bin=lambda x: np.floor(x.mag)
).pivot_table(
    index='mag_bin', columns='magType', values='mag', aggfunc='count'
)

pivot.plot.bar(
    stacked=True, rot=0,
    title='Earthquakes by integer magnitude and magType'
)

plt.ylabel('earthquakes') # label the axes (discussed in chapter 6)
```

```
Text(0, 0.5, 'earthquakes')
```

```
plt.show()
```

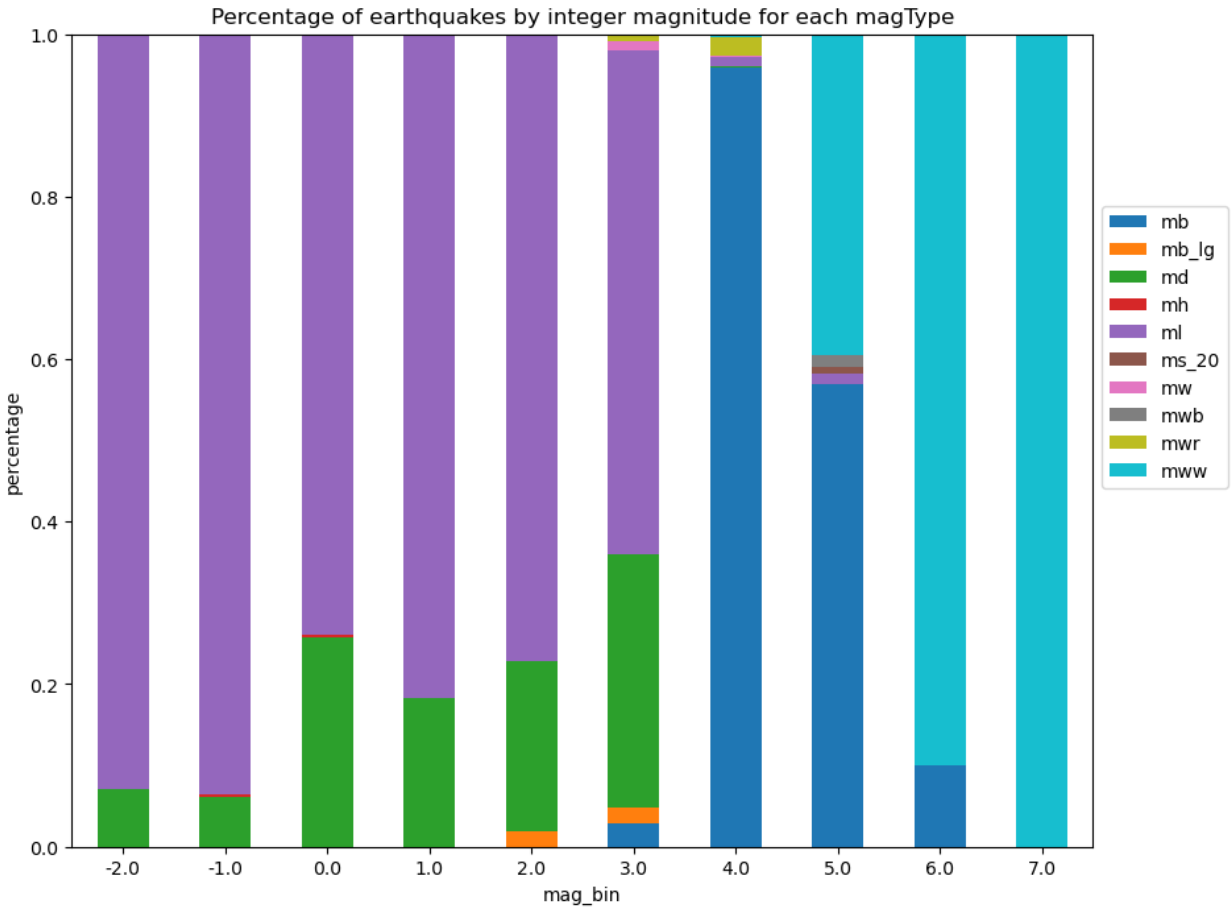


Normalized stacked bars

```
normalized_pivot = pivot.fillna(0).apply(lambda x: x/x.sum(), axis=1)
ax = normalized_pivot.plot.bar(
    stacked=True, rot=0, figsize=(10, 5),
    title='Percentage of earthquakes by integer magnitude for each
magType'
)
ax.legend(bbox_to_anchor=(1, 0.8)) # move legend to the right of the
plot
plt.ylabel('percentage') # label the axes (discussed in chapter 6)
```

```
Text(0, 0.5, 'percentage')
```

```
plt.show()
```



pandas plotting subpackage

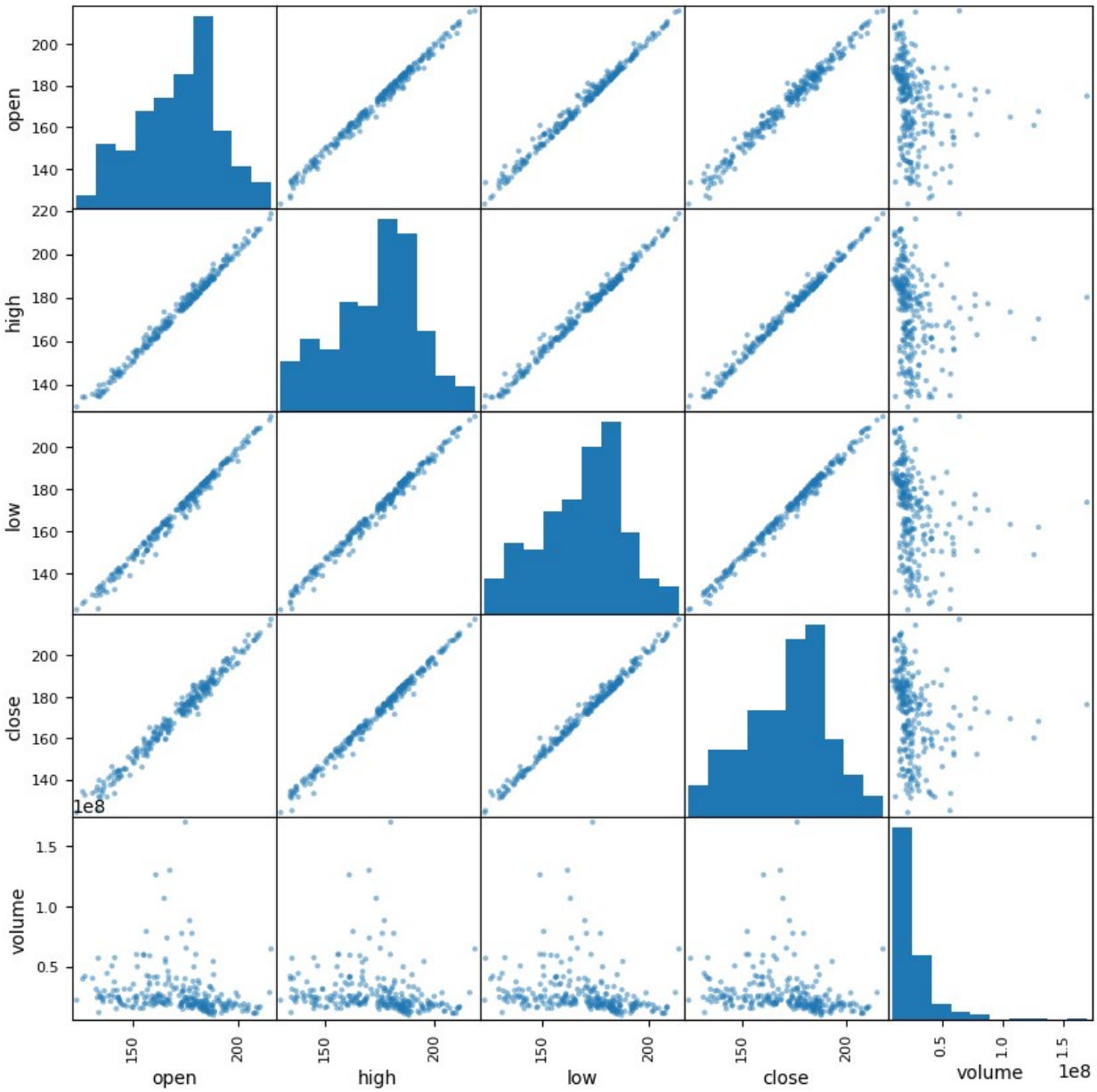
Scatter matrix

```
from pandas.plotting import scatter_matrix
scatter_matrix(fb, figsize=(10, 10))
```

```
array([[<Axes: xlabel='open', ylabel='open'>,
        <Axes: xlabel='high', ylabel='open'>,
        <Axes: xlabel='low', ylabel='open'>,
        <Axes: xlabel='close', ylabel='open'>,
        <Axes: xlabel='volume', ylabel='open'>],
       [<Axes: xlabel='open', ylabel='high'>,
        <Axes: xlabel='high', ylabel='high'>,
        <Axes: xlabel='low', ylabel='high'>,
        <Axes: xlabel='close', ylabel='high'>,
        <Axes: xlabel='volume', ylabel='high'>],
       [<Axes: xlabel='open', ylabel='low'>,
        <Axes: xlabel='high', ylabel='low'>,
        <Axes: xlabel='low', ylabel='low'>,
```

```
        <Axes: xlabel='close', ylabel='low'>,
        <Axes: xlabel='volume', ylabel='low'>],
       [<Axes: xlabel='open', ylabel='close'>,
        <Axes: xlabel='high', ylabel='close'>,
        <Axes: xlabel='low', ylabel='close'>,
        <Axes: xlabel='close', ylabel='close'>,
        <Axes: xlabel='volume', ylabel='close'>],
       [<Axes: xlabel='open', ylabel='volume'>,
        <Axes: xlabel='high', ylabel='volume'>,
        <Axes: xlabel='low', ylabel='volume'>,
        <Axes: xlabel='close', ylabel='volume'>,
        <Axes: xlabel='volume', ylabel='volume'>]], dtype=object)
```

```
plt.show()
```

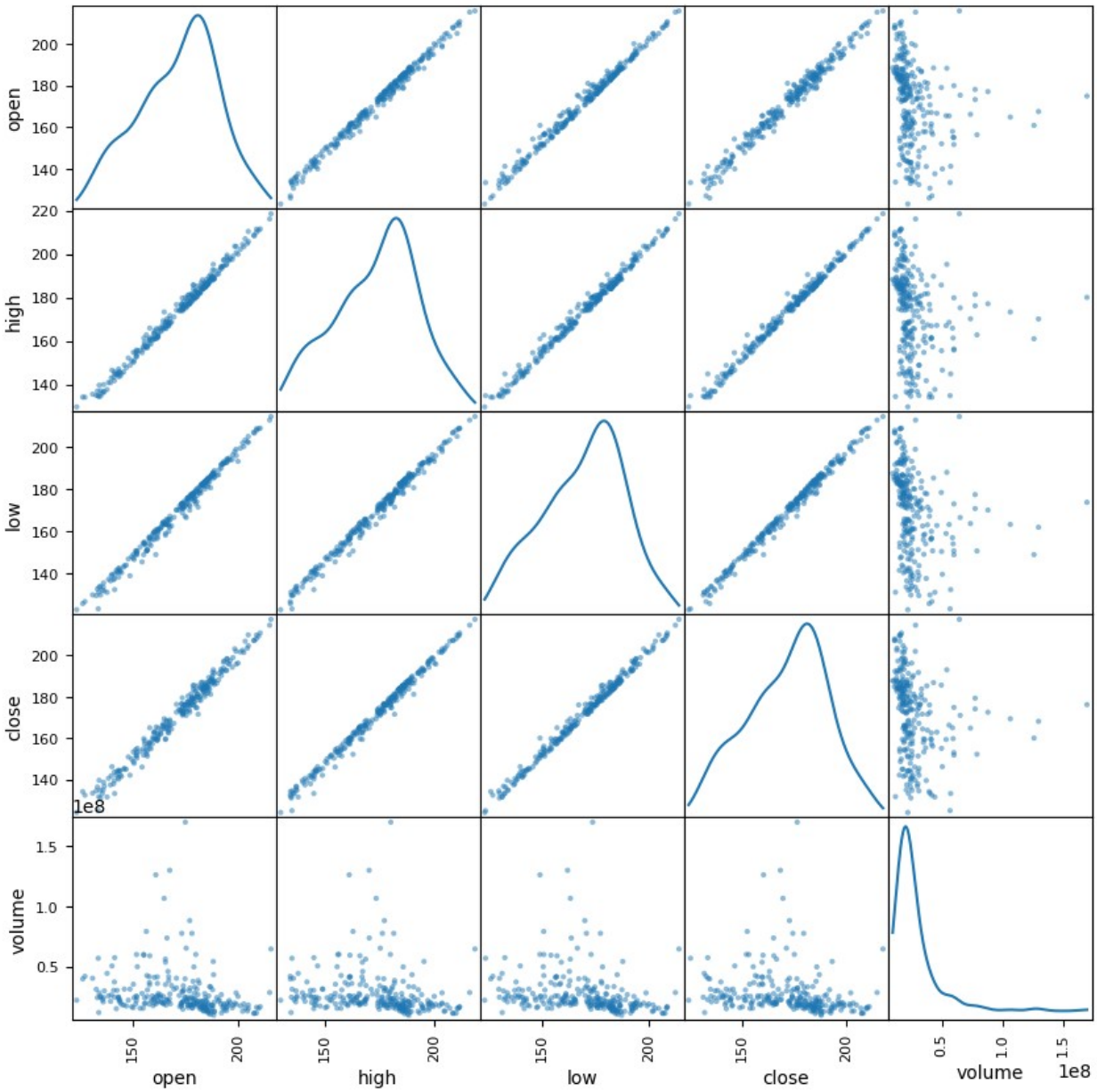



```
scatter_matrix(fb, figsize=(10, 10), diagonal='kde')
```

```
array([[<Axes: xlabel='open', ylabel='open'>,
        <Axes: xlabel='high', ylabel='open'>,
        <Axes: xlabel='low', ylabel='open'>,
        <Axes: xlabel='close', ylabel='open'>,
        <Axes: xlabel='volume', ylabel='open'>],
       [<Axes: xlabel='open', ylabel='high'>,
        <Axes: xlabel='high', ylabel='high'>,
        <Axes: xlabel='low', ylabel='high'>,
        <Axes: xlabel='close', ylabel='high'>,
        <Axes: xlabel='volume', ylabel='high'>],
```

```
       [<Axes: xlabel='open', ylabel='low'>,
        <Axes: xlabel='high', ylabel='low'>,
        <Axes: xlabel='low', ylabel='low'>,
        <Axes: xlabel='close', ylabel='low'>,
        <Axes: xlabel='volume', ylabel='low'>],
       [<Axes: xlabel='open', ylabel='close'>,
        <Axes: xlabel='high', ylabel='close'>,
        <Axes: xlabel='low', ylabel='close'>,
        <Axes: xlabel='close', ylabel='close'>,
        <Axes: xlabel='volume', ylabel='close'>],
       [<Axes: xlabel='open', ylabel='volume'>,
        <Axes: xlabel='high', ylabel='volume'>,
        <Axes: xlabel='low', ylabel='volume'>,
        <Axes: xlabel='close', ylabel='volume'>,
        <Axes: xlabel='volume', ylabel='volume'>]], dtype=object)
```

```
plt.show()
```

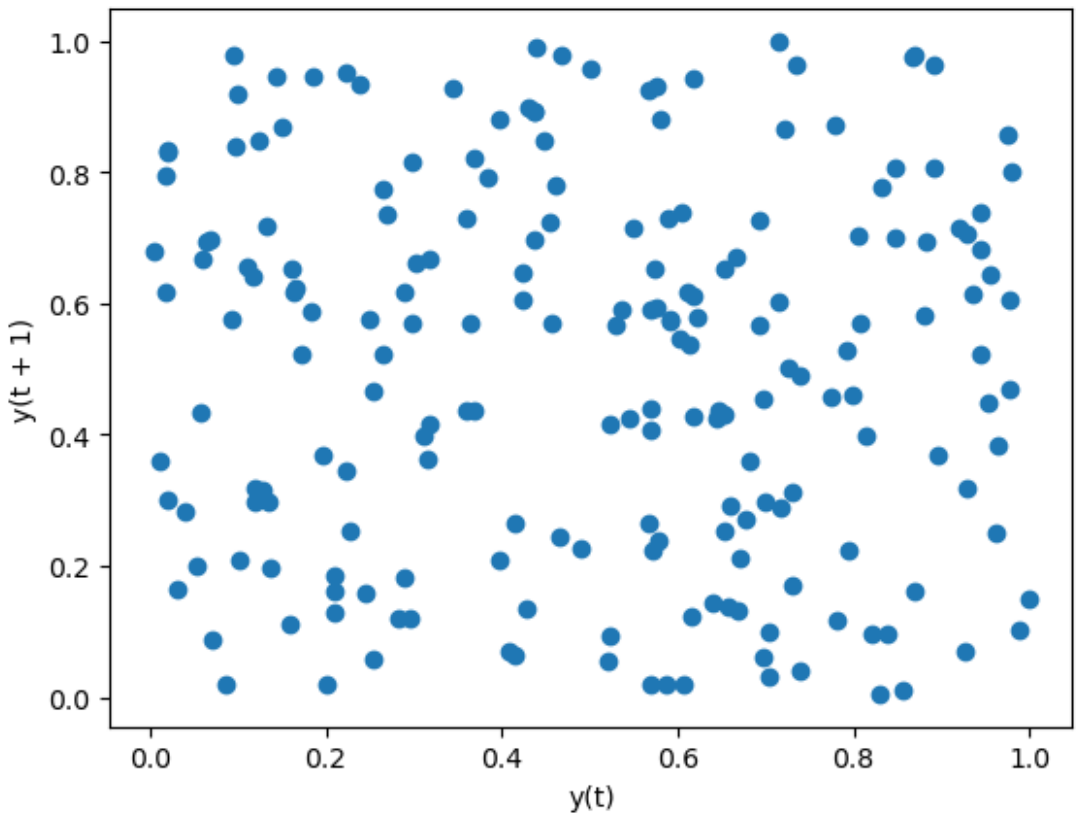


Lag plot

```
from pandas.plotting import lag_plot
np.random.seed(0) # make this repeatable
lag_plot(pd.Series(np.random.random(size=200)))
```

```
<Axes: xlabel='y(t)', ylabel='y(t + 1)'>
```

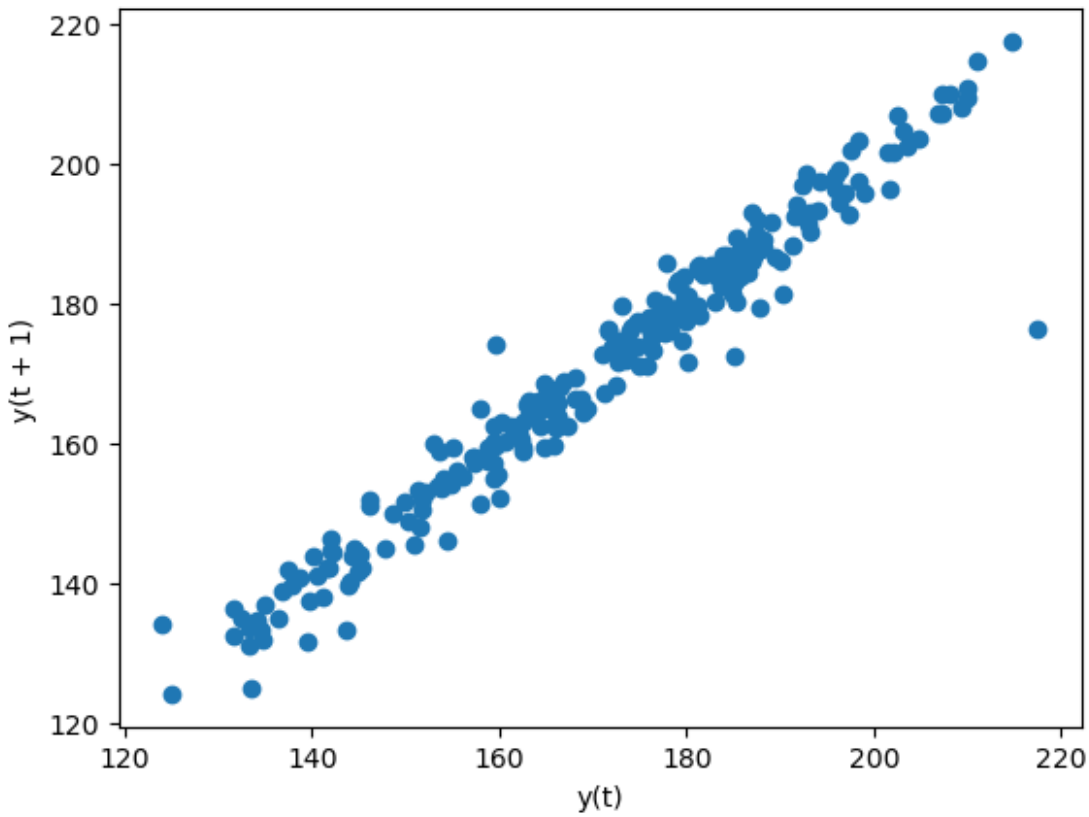
```
plt.show()
```




```
lag_plot(fb.close)
```

```
<Axes: xlabel='y(t)', ylabel='y(t + 1)'>
```

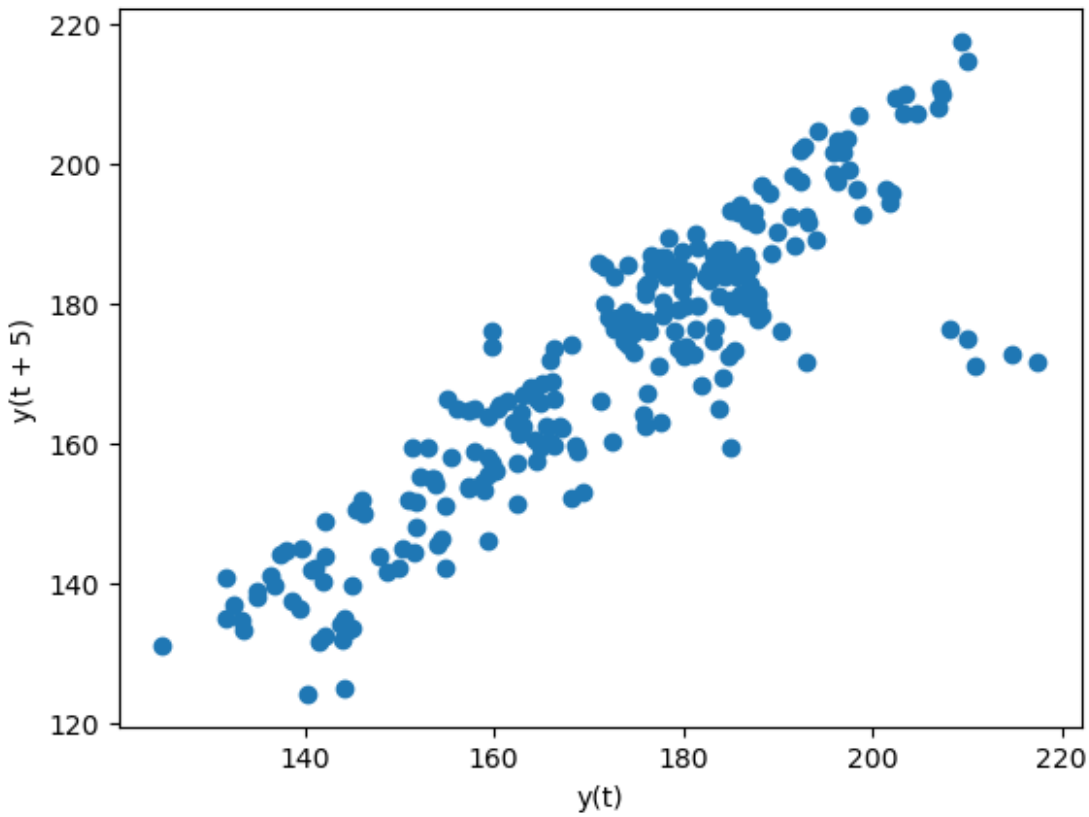
```
plt.show()
```




```
lag_plot(fb.close, lag=5)
```

```
<Axes: xlabel='y(t)', ylabel='y(t + 5)'>
```

```
plt.show()
```

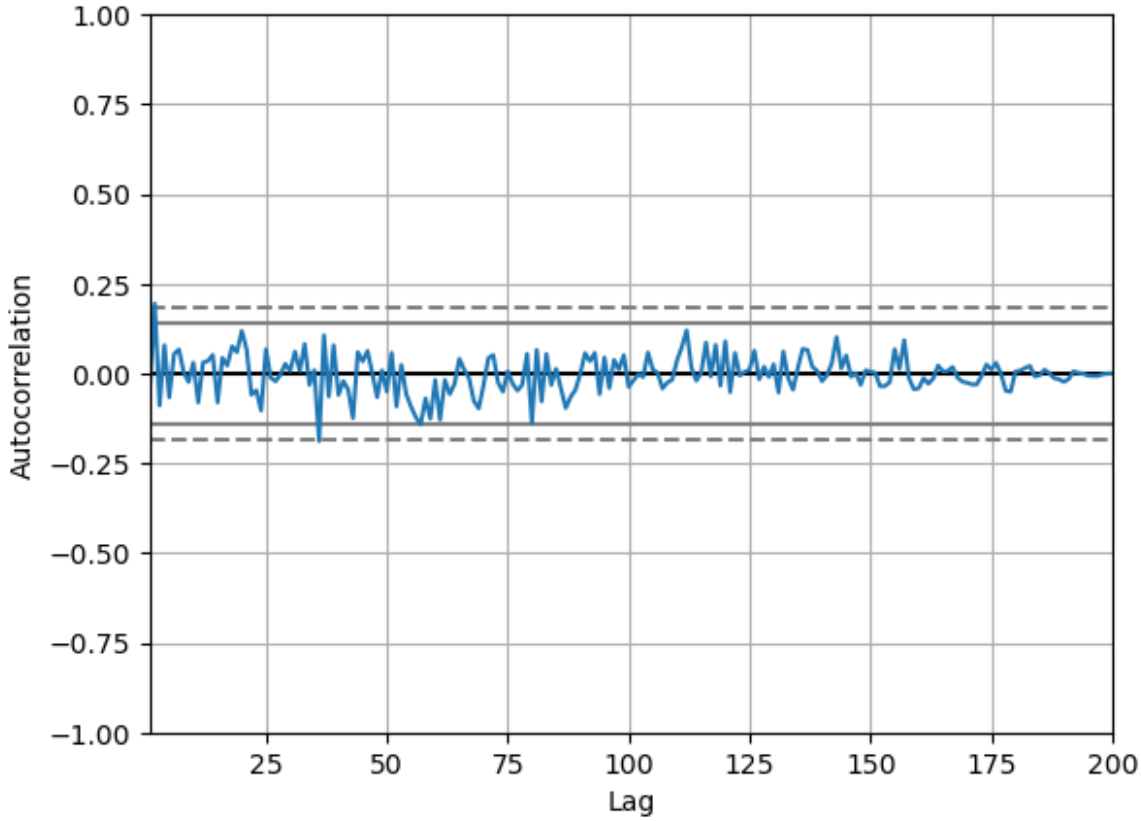


Autocorrelation plots

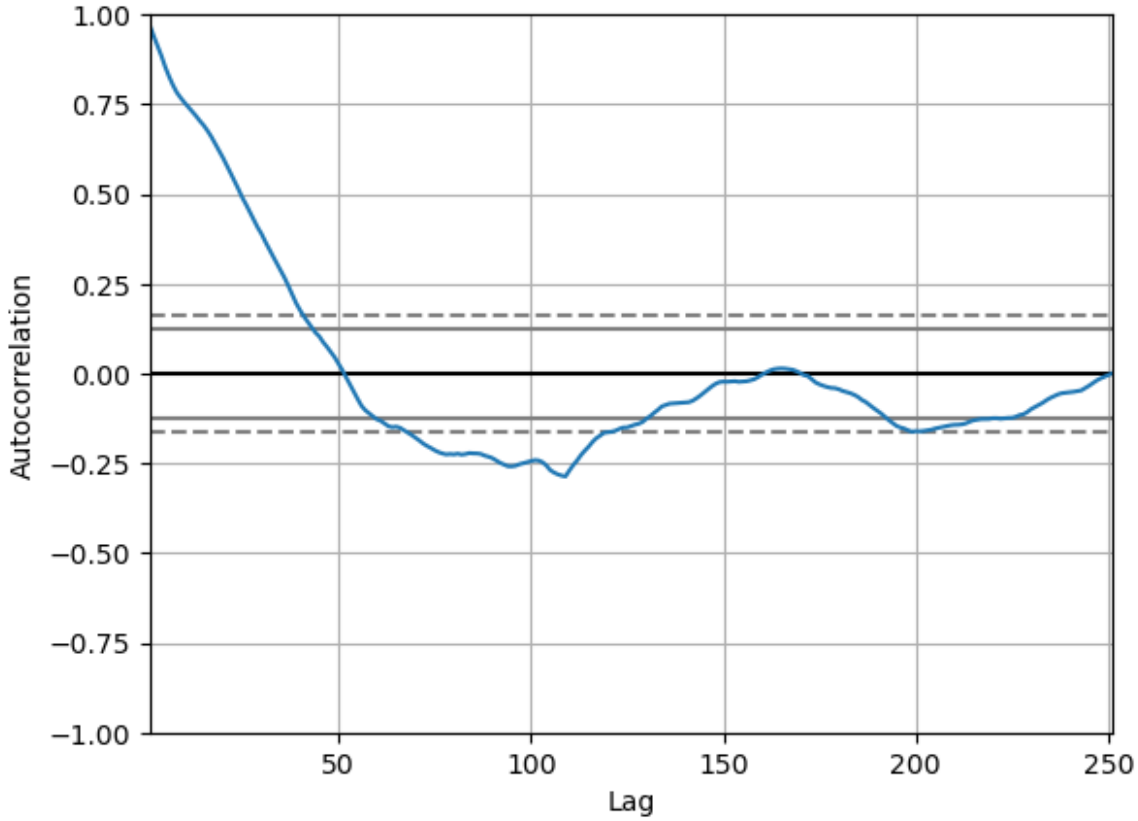
```
from pandas.plotting import autocorrelation_plot
np.random.seed(0) # make this repeatable
autocorrelation_plot(pd.Series(np.random.random(size=200)))
```

```
<Axes: xlabel='Lag', ylabel='Autocorrelation'>
```

```
plt.show()
```

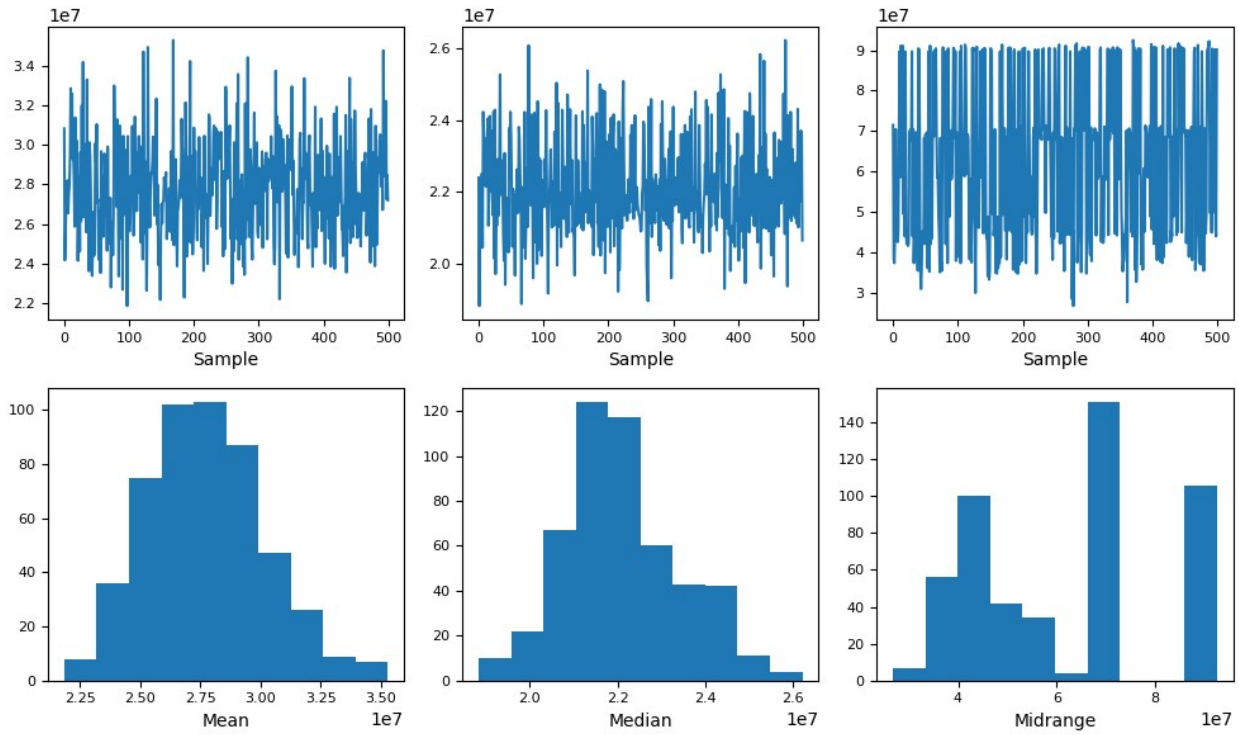



```
autocorrelation_plot(fb.close)
<Axes: xlabel='Lag', ylabel='Autocorrelation'>
plt.show()
```



Bootstrap plot

```
from pandas.plotting import bootstrap_plot
fig = bootstrap_plot(fb.volume, fig=plt.figure(figsize=(10, 6)))

plt.show()
```



Supplementary Activity

Using the CSV files provided and what we have learned so far in this module complete the following exercises:

```
fb.head()

               open    high       low   close    volume
date
2018-01-02  177.68  181.58  177.5500  181.42  18151903
2018-01-03  181.88  184.78  181.3300  184.67  16886563
2018-01-04  184.90  186.21  184.0996  184.33  13880896
2018-01-05  185.59  186.90  184.9300  186.85  13574535
2018-01-08  187.20  188.90  186.3300  188.28  17994726

quakes.head()

    mag magType           time                  place  tsunami
parsed_place
0  1.35      ml  1539475168010  9km NE of Aguanga, CA        0
California
1  1.29      ml  1539475129610  9km NE of Aguanga, CA        0
California
2  3.42      ml  1539475062610  8km NE of Aguanga, CA        0
California
3  0.44      ml  1539474978070  9km NE of Aguanga, CA        0
```

```
California
4  2.16      md  1539474716050  10km NW of Avenal, CA        0
California
```

1. Plot the rolling 20-day minimum of the Facebook closing price with the pandas plot() method.

```
fb['date'] = pd.to_datetime(fb['date'])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File c:\Users\Aldous\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3641, in Index.get_loc(self, key)
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'date'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[133], line 1
----> 1 fb['date'] = pd.to_datetime(fb['date'])

File c:\Users\Aldous\anaconda3\Lib\site-packages\pandas\core\frame.py:4378, in DataFrame.__getitem__(self, key)
   4376 if self.columns.nlevels > 1:
   4377     return self._getitem_multilevel(key)
-> 4378 indexer = self.columns.get_loc(key)
   4379 if is_integer(indexer):
   4380     indexer = [indexer]

File c:\Users\Aldous\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3648, in Index.get_loc(self, key)
```

```
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will
raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-
raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'date'
```

Conclusion

End